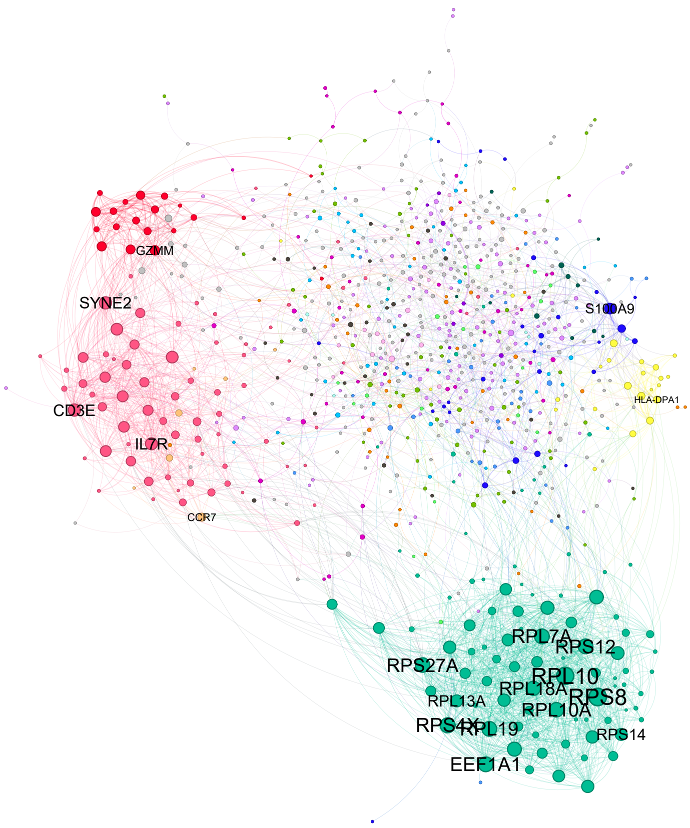
White Paper: An Architectural Framework for Reproducible and Donor-Aware scRNA-seq Systems (PBMC)



A Reference Implementation and Production-Grade Workflow using the 10x PBMC Dataset

Simo Inkala
Systems Biology Architect & Method Developer

March 2026

https://github.com/Inkasimo/scrnaseq-pbmc-workflow

https://doi.org/10.5281/zenodo.19021322

inkasimo@gmail.com

Contents

1 Executive Summary: Addressing the Reproducibility Crisis in Single-Cell Systems

- Dependency Hardening: Zero-drift environment through Docker digest-pinning and R-package vendoring.
- Auditability: Automated provenance tracking via Snakemake atomic execution sentinels.
- Validated Consistency: Implementation of Equivalence Testing (TOST) to confirm cross-donor conservation.

This project implements a reproducible, containerized single-cell RNA-seq (scRNA-seq) analysis workflow using Snakemake, Docker, and a Python-based CLI wrapper. The pipeline is demonstrated on publicly available 10x Genomics PBMC datasets (donors 1–4) and covers the full path from raw FASTQs to downstream and statistical modeling

The primary objective is to demonstrate disciplined workflow engineering: explicit execution structure, strict dependency control, restart-safe modular design, and transparent donor-aware statistical modeling. Reproducibility is treated as a first-class constraint through containerization, version-locked R environments, and digest-pinned runtime images.

While the biological results are internally coherent and consistent with expected PBMC structure, the emphasis of this project is methodological rather than discovery-driven. The repository is therefore intended as a technical portfolio artifact showcasing the design and implementation of a complete, auditable, and reproducible scRNA-seq analysis system.

1.1 Execution environment

Hardware:	
CPU	Intel Core Ultra 9 185H (16 cores, 22 threads)
RAM	32 GB
Storage	NVMe SSD: 1 TB (OS) + NVMe SSD: 2 TB (data)

All computations were performed on a local workstation running Windows 11 with WSL2 and Docker Desktop. The pipeline was executed entirely inside Docker containers to ensure environment isolation and reproducibility across hosts.

Due to the memory requirements of STAR genome indexing, the default WSL2 configuration was insufficient and resulted in out-of-memory (OOM) failures. The WSL2 memory limit was therefore explicitly increased to 32 GB. A reference configuration file is provided at `docs/wslconfig.example` to document the required settings and facilitate reproducible execution on similar systems.

2 System Architecture and Structural Design

The framework is engineered to enforce a strict decoupling of **Execution Logic**, **Orchestration Configuration**, and **Dependency Environments**. This architecture adheres to industry standards for high-integrity computational workflows, ensuring a clear boundary between upstream data engineering and downstream biostatistical inference.

- `containers/`
  Immutable Environment Layer: Contains the Docker build context defining a bit-identical execution environment. By utilizing `renv` and digest-pinning, this layer eliminates "environment drift" and ensures long-term reproducibility across heterogeneous HPC or Cloud infrastructure.

- `workflow/`
  Orchestration Logic: A Snakemake-based Directed Acyclic Graph (DAG) that encodes the full provenance of the analysis. It manages the lifecycle of FASTQ acquisition, STARsolo alignment, and multi-stage QC gates.

- `config/`
  Governance & Parameters: Centralized YAML-based configuration controlling dataset definitions and algorithmic thresholds. This ensures that the "Scientific Logic" is separated from the "Code Logic," allowing for rapid, auditable parameter sweeps.

- `resources/`
  Hardened Dependency Assets: Local, versioned copies of critical external metadata (e.g., 10x Whitelists, MSigDB gene sets). This "Dependency Hardening" strategy removes reliance on unstable external URLs and ensures audit-ready execution in air-gapped or restricted environments.

- `data/`
  Managed Data Tier: Structured storage for raw telemetry (FASTQs) and genomic reference assets. This directory is treated as a read-only input source once the pipeline initiates.

- `results/`
  Audit Trail & Deliverables: A comprehensive output tree including structured logs, STARsolo telemetry, and Seurat-serialized objects. Each artifact is coupled with an execution log to ensure full technical provenance.

- `docs/`
  Knowledge Base: Technical specifications, implementation white papers (including this document), and "Golden Set" example outputs for pipeline validation.

- `scripts/`
  Analytical Kernels: Specialized R-modules for pseudobulk modeling, **TOST-based equivalence testing**, and systems-level network inference.

- `run_analysis.py`
  High-Level Execution Controller: A Python-based CLI wrapper that serves as the primary interface. It orchestrates the Snakemake DAG within the Docker container, enforcing modular execution and providing a simplified "Control Plane" for the end-user.

3 R Environment and Reproducibility Strategy

3.1 Overview

All R-based analyses in this workflow are executed inside a Docker container with a fully specified and reproducible R environment.

Dependency management is handled using `renv`, while containerization ensures isolation from host-specific configurations. This design minimizes environment drift and allows consistent re-execution across machines and over time.

3.2 R Version and Package Management

The container runtime is pinned to a fixed R base image (`rocker/r-ver:4.3.3`). R package dependencies are declared in a version-locked `renv.lock` file, and package resolution is pinned to a CRAN snapshot via the environment variable:
`RENV_CONFIG_REPOS_OVERRIDE=https://packagemanager.posit.co/cran/2024-01-01`.

The lockfile captures:

- The R version used (R 4.3.3)
- The CRAN snapshot date (`https://packagemanager.posit.co/cran/2024-01-01`)
- The exact versions and sources of all R packages required for the analysis

During the Docker image build, the full R environment is restored non-interactively using `renv::restore()`. The restored library is installed into a fixed path inside the container (`/opt/renv/library`) via `RENV_PATHS_LIBRARY`, and no R package installation or snapshotting occurs at runtime.

Bioconductor resolution is explicitly pinned during the image build (`Bioconductor 3.18`) to ensure consistent installation of Bioconductor packages.

3.3 Supply Chain Hardening: Mitigation of Upstream Dependency Risk

The core analysis depends on `Seurat` [1] and `SeuratObject` [2], which are fast-evolving packages and common sources of reproducibility failure due to upstream repository volatility or "breaking" API changes.

By implementing a **vendoring strategy**, the framework ensures immunity to external repository state—a critical requirement for GxP-compliant computational environments and long-term auditability.

To mitigate this risk, specific, validated versions are archived directly within the repository structure:

- `renv/vendor/seurat-v4.4.0.tar.gz`
- `renv/vendor/seurat-object-v4.1.4.tar.gz`

During the Docker build phase, these local sources are injected into the container (`COPY renv/vendor/ /work/renv/vendor/`) and prioritized by `renv::restore()` as the authoritative package sources recorded in the `renv.lock` manifest.

This architecture provides three primary engineering safeguards:

- **Decoupled Build Lifecycle:** Docker image construction is independent of GitHub uptime or the transient state of external branches.
- **Immutable Dependency State:** The analytical environment is shielded from upstream CRAN/Bioconductor version drift, preventing "silent failures" during re-execution.
- **Source Integrity:** The exact source code utilized for the inference is preserved as a first-class citizen of the project, facilitating full forensic auditability of the results.

4 Docker Integration

The Docker image is built from `rocker/r-ver:4.3.3`, providing a fixed R runtime on a Debian-compatible base. System-level dependencies required for R packages and external tools (e.g. STAR, FastQC) are installed explicitly during the image build.

The image also configures explicit `renv` paths to avoid host-dependent behavior:

- `RENV_PATHS_LIBRARY=/opt/renv/library`
- `RENV_PATHS_CACHE=/opt/renv/cache`
- `RENV_CONFIG_CACHE_SYMLINKS=FALSE`

This ensures that both the installed packages and the renv cache are fully contained within the image filesystem and are not affected by host-side caches or symlinks.

All R packages are restored during image build using `renv::restore()`. The Docker build does not modify the lockfile; dependency changes must be made explicitly by updating `renv.lock` and committing it to version control.

4.1 Docker Image Distribution

A pre-built Docker image corresponding to the archived workflow environment is published to GitHub Container Registry (GHCR). The versioned release can be pulled using:

```
docker pull ghcr.io/inkasimo/scrnaseq-pbmc-workflow:v2.0.0
```

For strict archival reproducibility, the digest-pinned image corresponding exactly to the environment used to generate the archived results can be pulled using:

```
docker pull ghcr.io/inkasimo/scrnaseq-pbmc-workflow@sha256:
a486aee53be7f89ebf3305f992bbc9a9103b40345f4a9791022324f3d2a5f747
```

Local image builds (`docker build`) are supported for development purposes but are not guaranteed to be bit-identical to the archived image due to potential upstream system library changes.

4.2 Separation of Concerns

- Host system: used primarily to edit code and commit changes. Dependency updates are captured by regenerating and committing `renv.lock`. In practice, lockfile updates are performed by launching an interactive R session inside the Docker container used for development, then committing the resulting lockfile on the host.
- Docker image (execution): consumes the lockfile in a read-only fashion during image build and runtime, providing a stable and reproducible execution environment.
- IDE tooling: automatic IDE-driven package installation is disabled to prevent contamination of the project dependency set.

Result

This setup ensures:

- A pinned and auditable R environment (R version, CRAN snapshot, and package sources)
- Long-term stability of critical dependencies (notably Seurat via vendoring)
- Clear provenance of all software components
- Safe integration with Snakemake and container-based execution

In summary, the combination of `renv`, vendored critical packages, and Docker provides a robust and auditable foundation for reproducible single-cell RNA-seq analysis.

4.3 Windows / WSL / Docker Desktop

This workflow runs inside Docker and bind-mounts the repository into the container. On Windows, Docker Desktop must have access to the repository path; otherwise the container will not be able to read files under `/work`.

If the following type of error is encountered:

```
ERROR: Current working directory no longer exists.  This can happen if
you deleted or recloned the repository, or if WSL/Docker mount state became
inconsistent, or if Docker cannot run at all (e.g.  corporate-managed machine
with restricted virtualization).  Try restarting WSL and cd into the repository
root.
```

Docker Desktop file-sharing settings should be checked for the drive containing the repository. Bind-mount restrictions are common on managed or corporate machines; in such cases, running the workflow on Linux or macOS avoids these limitations.

Restarting WSL may help

5 Operational Logic and Pipeline Orchestration

The pipeline is orchestrated using Snakemake and executed inside Docker containers to ensure reproducibility and avoid host-specific environment drift.

The workflow is implemented as a `workflow/Snakefile` (the authoritative DAG and rule logic) and an optional Python wrapper (`run_analysis.py`) that provides a safer, section-based CLI for running the workflow inside the Docker image.

Conceptually, users select a "section" (e.g. QC, alignment, downstream), the wrapper maps that intent to explicit Snakemake targets (mostly `*.done` sentinel files), and Snakemake computes and executes the minimal set of rules required to produce those targets.

5.1 Snakemake workflow (`workflow/Snakefile`)

The Snakefile encodes the pipeline as a directed acyclic graph (DAG) of rules, where each rule declares explicit `input`, `output`, and execution logic (shell or Python `run` blocks).

A core design choice is the use of lightweight sentinel "done" files (e.g. `fastqc.done`, `starsolo.done`, `seurat_qc.done`) as rule outputs. These serve two purposes:

- They provide a stable, human-auditable completion marker for each stage without requiring Snakemake to hash or reason over large binary outputs.
- They make the workflow resilient to partial outputs: a rule is only considered complete when its sentinel file is created after all expected artifacts have been written successfully.

Sentinels are created atomically (write `.tmp` then move/replace), which reduces the chance of leaving a misleading "complete" marker after an interrupted job.

The workflow also supports two execution branches: `raw` (default) and `trimmed` (optional). Trimming is applied only to Read 2 and, if enabled, all downstream steps automatically route to the trimmed branch while preserving the on-disk STARsolo layout (legacy mapping of `untrimmed` → `raw`).

Configuration is driven by `config/config.yaml`, with CLI overrides supported for key toggles (e.g. download FASTQs, build STAR index, enable trimming). Rules are written to be explicit about required resources (threads, reference paths, gene sets, marker sets), and logs are captured under `results/logs/` for auditability.

5.2 High-Level Execution Controller (run_analysis.py)

While the Snakefile can be executed directly inside Docker, most users run the pipeline via `run_analysis.py`, which wraps Snakemake in a section-based CLI.

The wrapper's main job is to translate high-level user intent into explicit Snakemake targets. Each CLI "section" corresponds to a curated list of target files (primarily `*.done` sentinels and a small number of concrete outputs such as `multiqc_report.html`). Snakemake then builds exactly what is required to satisfy those targets, and nothing else.

This approach improves usability and safety:

- **Section-based execution:** users run a named stage (`qc`, `align`, `downstream`, ...) instead of managing raw Snakemake targets.
- **Prevents accidental full runs:** the wrapper requires selecting a section, rather than implicitly executing the full `rule all`.
- **Mode toggles are explicit:** flags like `--trimmed` and `all_no_download` translate into controlled Snakemake config overrides (e.g. `trim_enabled=true`, `download_fastqs=false`).
- **Docker execution is standardized:** the wrapper always executes Snakemake inside the pinned container image, bind-mounting the repository and passing consistent CPU/thread settings.
- **Early failure checks:** the wrapper validates that Docker bind mounts can see `workflow/Snakefile`, avoiding confusing "Snakefile not found" failures caused by file-sharing restrictions on some Windows/WSL setups.

The wrapper also exposes convenience commands for discovery (`--list-sections`, `--list-donors`) and supports passing through advanced Snakemake options via `--extra` for power users.

5.3 Available wrapper sections

The full user manual documents all sections and examples; for quick reference, the wrapper exposes the following top-level sections:

- `download_data`
- `download_data_and_qc`
- `qc`
- `ref`
- `trim`
- `trim_and_qc`
- `align`
- `upstream`
- `upstream_no_download`
- `build_seurat_object_qc`
- `filter_and_normalize_seurat`
- `cluster_annotate_seurat`
- `deg_and_tost`
- `networks`
- `downstream`
- `all`
- `all_no_download`
- `unlock`

Internally, the wrapper maps these user-facing sections onto specific target sets (for example, `qc` expands to per-donor FastQC sentinels plus the aggregated MultiQC report). This design keeps the workflow modular and transparent while making execution approachable for users who do not want to interact directly with Snakemake DAG mechanics.

6 Data

Raw FASTQ files are obtained from publicly available 10x Genomics Chromium X Series datasets corresponding to the 5k Human PBMC 3′ Gene Expression (GEM-X v4) libraries for Donors 1–4. Each donor is provided as a separate FASTQ archive and downloaded directly from the 10x Genomics data portal.

The samples consist of peripheral blood mononuclear cells (PBMCs) isolated from healthy human donors (male, age 18–35), prepared by 10x Genomics and sourced from Cellular Technologies Limited. Cells were processed using the Chromium GEM-X Single Cell 3′ Reagent Kits v4 chemistry and sequenced on an Illumina NovaSeq 6000 platform at approximately 39,000 read pairs per cell.

Libraries were generated as paired-end, dual-indexed runs with the following configuration: 28 cycles for Read 1 (cell barcode + UMI), 10 cycles for i7 index, 10 cycles for i5 index, and 90 cycles for Read 2 (cDNA insert). Read 1 therefore encodes the 16 bp cell barcode and 12 bp UMI, while Read 2 contains the transcript-derived sequence used for alignment and quantification.

The datasets were originally processed by 10x Genomics using the `cellranger multi` pipeline, and automated cell type annotations were generated using the `human_pca_v1_beta` reference model. **However, in this project, only the raw FASTQ files are used; all alignment and downstream analyses are performed independently within the present workflow.**

Each donor dataset contains approximately 5,000–6,000 recovered cells and represents cryopreserved human PBMC suspensions. The data are licensed under the Creative Commons Attribution 4.0 International (CC BY 4.0) license and are publicly accessible through the 10x Genomics website.

Companion datasets include the individual 5k PBMC datasets for Donors 1–4 and a 20k multiplexed PBMC dataset combining all donors; however, this workflow analyzes the donors independently.

7 Toy Demonstration Dataset and Mini Reference

To enable a fast, resource-light demonstration of the workflow, a self-contained toy dataset was constructed. The objective of the toy configuration is to allow users to execute the full upstream pipeline (QC → alignment → Seurat object construction) within approximately 5–10 minutes on a standard workstation.

The toy configuration intentionally terminates after Seurat object construction and QC. Downstream normalization, clustering, differential expression is disabled in toy mode.

7.1 FASTQ Subsampling

A reduced read set was generated from the publicly available 10x Genomics PBMC data by truncating the original FASTQ files to a fixed number of reads per mate. This preserves realistic read structure (16 bp cell barcode + 12 bp UMI) while dramatically reducing runtime.

The toy configuration uses:

- Single donor
- 100,000 read pairs
- Untrimmed execution branch

This preserves realistic 10x geometry while keeping alignment computationally lightweight.

7.2 Mini Reference Construction

Using the full GRCh38 reference for the toy demonstration would require tens of gigabytes of RAM for index loading. To ensure broad usability, a chromosome-restricted reference was created.

Chromosome-restricted FASTA Chromosome 1 was extracted from the primary assembly:

```
samtools faidx GRCh38.primary_assembly.genome.fa chr1 \
  > GRCh38.chr1.fa
```

Chromosome-restricted GTF The annotation was filtered to retain header lines and chr1 records only:

```
awk 'BEGIN{FS="\t"} /^#/ || $1=="chr1"' \
  gencode.v45.annotation.gtf \
  > gencode.v45.chr1.gtf
```

STAR Index Generation For the current toy configuration, the STAR index is **not** precomputed or distributed. Instead, the index is generated dynamically during execution of the toy workflow:

```
STAR \
  --runMode genomeGenerate \
  --genomeDir data/ref/toy/star_index_chr1 \
  --genomeFastaFiles GRCh38.chr1.fa \
  --sjdbGTFfile gencode.v45.chr1.gtf \
  --sjdbOverhang 89
```

Building the chromosome-restricted index at runtime requires only modest memory and typically completes within 2–5 minutes on a standard workstation. This approach significantly reduces the size of the distributed toy bundle while preserving realistic alignment behavior.

7.3 Design Rationale

The toy dataset is intentionally separated from the full production configuration:

- Toy demo: chromosome-restricted reference, STAR index built during execution, fast execution.
- Full archive: complete GRCh38 + GENCODE v45 index prebuilt for production-scale analysis.

Both are distributed under the same Zenodo concept record to ensure reproducibility and long-term archival stability. This separation reflects a deliberate engineering choice: demonstration constraints are distinct from production-scale analysis requirements.

8 Quality Control

Raw FASTQ quality was assessed using FastQC v0.11.9 and summarized with MultiQC v1.33. QC results were used to evaluate the necessity of adapter trimming prior to alignment.

Across donors and lanes, base quality profiles are high and stable over the read length, and the aggregated MultiQC status table reports `PASS` for R2 *Adapter Content*, indicating no evidence of pervasive adapter contamination in the cDNA read.

The primary flagged item is *Overrepresented sequences*, most notably in Donor 4 R2 files. The top sequences occur at approximately FastQC's reporting threshold and are consistent with known short-oligo / library-structure artifacts (e.g., template-switch / SMARTer-like carryover) rather than systematic adapter read-through affecting a substantial fraction of reads.

For 10x Genomics $3'$ Gene Expression data, explicit trimming is typically unnecessary and can be counterproductive: standard single-cell pipelines (e.g., Cell Ranger and STARsolo) rely on preserving the expected read structure, and modest $3'$ artifacts are generally handled via soft-clipping during alignment without compromising UMI counting. Therefore, trimming was omitted for the main analysis and is only enabled if strong adapter signal is observed in QC.

The workflow supports an optional trimming branch via a dedicated wrapper section (and corresponding Snakemake rules), but this branch was not activated for the reported results given the QC outcome (see Section **Read Trimming (cutadapt)**).

All QC reports are written to `results/qc/` (FastQC per-donor outputs and aggregated MultiQC reports), with execution logs stored under `results/logs/qc/`.

9 Read trimming (cutadapt)

The workflow supports an optional trimming branch implemented with `cutadapt` and exposed through dedicated wrapper sections (e.g. `trim`, `trim_and_qc`) and the `-trimmed` flag. When activated, all downstream steps (QC, alignment, and analysis) resolve against trimmed FASTQs and write into a separate `trimmed/` results branch.

Trimming is applied to Read 2 (cDNA) only; Read 1 (cell barcode + UMI) is never sequence-modified. Read pairs are filtered jointly using `-pair-filter=any`, so pairs are discarded if R2 fails the minimum-length threshold.

9.1 Configuration

Trimming parameters are defined in `config/config.yaml` and passed into the Snakemake rule via `TRIM_R2_*` variables.

```
trim:
  enabled: false
  adapter_r2: AGATCGGAAGAG
  q_r2: 20
  minlen_r2: 60
```

In the reported trimming run, `cutadapt v4.9` was executed with `-A AGATCGGAAGAG`, `-Q 0,20`, and `-minimum-length 0:60`, i.e. an effective `minlen_r2` of 60.

9.2 Implementation details

Outputs are written lane-by-lane to temporary files and atomically published on success; the `trim.done` sentinel is created only after all lanes complete. This prevents partially written FASTQs from being consumed downstream.

9.3 Observed impact

Across all donors, the proportion of read pairs discarded as too short after trimming was low and consistent (approximately 0.3–0.4%). This indicates minimal short-insert filtering and no evidence of pervasive adapter read-through.

Given the strong baseline QC metrics and the negligible trimming impact, downstream analyses in this report use the default untrimmed branch. The trimming pathway remains available but was not required for these datasets.

10 Alignment and Quantification

10.1 Choice of aligner: STARsolo vs Cell Ranger

Alignment and UMI-aware quantification were performed using `STARsolo` (STAR v2.7.11b) rather than Cell Ranger. The primary motivation is transparency and reproducibility: STARsolo exposes all alignment and barcode-handling parameters explicitly, integrates directly into Snakemake, and runs inside the pinned Docker environment.

This avoids reliance on opaque defaults or vendor-controlled pipelines while preserving compatibility with standard 10x data structures (gene–cell matrices equivalent to Cell Ranger output).

10.2 Reference genome and annotation

The reference was built locally from the following components:

- GRCh38 primary assembly
- GENCODE v45 gene annotation
- STAR v2.7.11b
- 10x GEM-X 3′ v4 barcode whitelist (bundled in `resources/barcodes/`)

The STAR index was generated inside the container using `-sjdbOverhang 89` (R2 length 90 bp), and is not distributed with the repository due to size. Index generation requires approximately 25–30 GB RAM.

10.3 Read structure and chemistry assumptions

Libraries correspond to 10x Chromium GEM-X 3′ v4 chemistry. Read structure assumptions were made explicit in the STARsolo invocation:

- R1: 16 bp cell barcode + 12 bp UMI (28 bp total)
- R2: cDNA (90 bp)
- `-soloType CB_UMI_Simple`
- `-soloCBlen 16`
- `-soloUMIlen 12`
- `-soloBarcodeReadLength 28`
- `-soloBarcodeMate 2`

Whitelist filtering was enforced via `-soloCBwhitelist`.

10.4 Counting and outputs

Gene-level UMI counting was performed internally by STARsolo using `-soloFeatures Gene`. Both raw and filtered gene–cell matrices were generated. No external counting tools (e.g. featureCounts) were used.

10.5 BAM output

BAM output was disabled via `-outSAMtype None` to reduce disk usage and memory overhead. STARsolo gene–cell matrices are complete without BAM files. BAM generation can be enabled if required for downstream inspection.

10.6 Execution environment

Alignment was executed inside Docker under WSL2. Memory was configured to allow STAR index generation and alignment:

```
[wsl2]
memory=32GB
processors=8
swap=16GB
```

No additional host-specific tuning was required.

10.7 Alignment summary statistics

Alignment metrics were consistent across donors. Unique mapping rates ranged from 68.8% to 72.4%, multi-mapping from 19.3% to 21.7%, and reads unmapped due to short length from 7.6% to 9.4%. Mismatch rates were stable at approximately 0.21–0.23%. No chimeric reads were reported.

Donor	Input Reads (M)	Unique (%)	Multi (%)	Unmapped Short (%)	Mismatch (%)
Donor1	227.1	71.89	20.18	7.83	0.23
Donor2	207.0	72.41	19.33	8.16	0.22
Donor3	185.7	72.42	19.84	7.64	0.21
Donor4	207.8	68.77	21.74	9.37	0.22

Table 1: STARsolo alignment summary statistics (raw branch).

Mapping performance is consistent with high-quality 10x $3'$ scRNA-seq libraries; donor4 shows a modestly lower unique mapping rate and higher short-read fraction but remains within expected bounds for PBMC datasets.

11 Downstream Analysis

11.1 Overview

Downstream analyses were performed on donor-specific expression datasets derived from STARsolo filtered gene–cell matrices. Seurat objects were used for quality control, normalization, clustering, and marker-based annotation. Pseudobulk differential expression was derived from the resulting QC-filtered and annotated data representations.

The workflow proceeds in five stages: (i) Seurat object construction from STARsolo outputs, (ii) donor-specific quality control and normalization, (iii) clustering and marker-based cell-type annotation, (iv) donor-aware pseudobulk differential expression with equivalence testing. All analyses were executed inside the containerized R environment to ensure full reproducibility.

All Seurat-based analyses were performed using `Seurat v4.4.0` and `SeuratObject v4.1.4` as specified in the `renv.lock` file embedded in the Docker image.

12 Methods – Statistical Framework and Signal Processing

12.1 Seurat object construction

Filtered STARsolo gene–barcode matrices were imported into Seurat. Feature names were set to gene symbols; symbol-less Ensembl placeholder rows were removed. Duplicate symbols were collapsed by summing counts to ensure a unique feature space per object.

12.2 Quality control filtering

Quality control was performed independently per donor using data-adaptive thresholds derived from empirical distributions of QC metrics.

Filtering rules:

- `percent.mt` $\leq \min(q_{90}, 25)$
- `nFeature_RNA` $\geq q_{05}$ and $\leq \min(q_{99}, 6000)$
- `nCount_RNA` $\geq q_{05}$ and $\leq q_{99}$
- `percent.hb` ≤ 1

For each donor, thresholds and filtering summaries were written to disk. Filtering decisions were therefore fully auditable.

12.3 Normalization and feature selection

QC-filtered objects were normalized using Seurat's `LogNormalize` method (library-size scaling followed by log-transformation; scale factor = 10,000). Highly variable genes (HVGs; $n = 2000$)

were identified using the VST method.

Scaling and regression of `percent.mt` were applied to HVGs for dimensionality reduction only. Scaled values were not used for differential expression.

12.4 Clustering and marker-based annotation

Dimensionality reduction was performed using PCA (first 30 components), followed by nearest-neighbor graph construction and Louvain clustering (resolution = 0.3). UMAP embeddings were generated using a fixed random seed.

Cell type annotation was performed using predefined marker gene sets. For each marker set, module scores were computed using `AddModuleScore`. Each cell was assigned the label corresponding to its highest module score. Cluster-level majority labels were also computed for reporting and downstream grouping. PBMC marker genes represented in table 2.

Cell type	**Marker genes**
T-cells	TRAC, CD3D, CD3E, LTB
CD4 T-cells	IL7R, CCR7, LTB, MALAT1
CD8 T-cells	CD8A, CD8B, NKG7, GZMK
NK-cells	NKG7, GNLY, PRF1, FCGR3A, XCL1
B-cells	MS4A1, CD79A, CD74, HLA-DRA
Plasma cells	MZB1, XBP1, JCHAIN
CD14 Monocytes	LYZ, S100A8, S100A9, LGALS3, CTSS
FCGR3A Monocytes	LYZ, FCGR3A, MS4A7, LGALS3
Dendritic cells	FCER1A, CST3, CLEC10A
Platelets	PPBP, PF4, NRGN

Table 2: Canonical PBMC marker genes used for manual cell-type annotation. Marker sets were curated from established PBMC single-cell RNA-seq references and workflows, including the 10x Genomics PBMC 3k/10k datasets [5], dendritic cell characterization by [6], and Seurat-based PBMC analyses [7, 8].

The annotation strategy intentionally favors transparency over automated reference mapping. Marker sets are small and interpretable, and labels are assigned directly from module score comparisons rather than external reference models. While this approach produces stable coarse PBMC lineage assignments, finer subtype distinctions or ambiguous cell states would benefit from complementary reference-based annotation methods (e.g., SingleR or Seurat reference mapping).

12.5 Pseudobulk differential expression and equivalence testing

Differential expression was performed at the donor level using a pseudobulk strategy.

Pseudobulk construction: For each donor and coarse immune group (table 3.), raw UMI counts were summed across cells (minimum 50 cells per donor × group). Donor was treated as the experimental unit.

DEG group	Constituent cell types
T-like	T-cells, CD4 T-cells, CD8 T-cells, NK-cells
B-like	B-cells, Plasma cells
Mono-like	CD14 Monocytes, FCGR3A Monocytes

Table 3: Cell-type groupings used for pseudobulk differential expression analysis.

Statistical model and differential expression: Pseudobulk differential expression was performed using `DESeq2` v1.42.1 (Bioconductor 3.18). Counts were modeled using the design:

$$\sim \texttt{donor} + \texttt{group},$$

thereby controlling for donor-specific baseline effects while testing cell-type group contrasts. Wald tests were used for coefficient inference, and $\log_2$ fold changes correspond to the estimated group effect. P-values were adjusted for multiple testing using the Benjamini–Hochberg procedure.

Contrasts:

Contrast	Comparison
C1	T-like vs B-like
C2	T-like vs Mono-like
C3	B-like vs Mono-like

Table 4: Pseudobulk differential expression contrasts evaluated using DESeq2.

Marker genes: Strict marker genes were defined as those satisfying

$$\texttt{padj} < 10^{-10} \quad \text{and} \quad |\log_2 \mathrm{FC}| > 3.$$

12.5.1 Equivalence testing (TOST):

Unlike standard differential expression, this framework prioritizes 'Bio-equivalence,' ensuring that identified gene programs are robust features of the biological system rather than donor-specific noise.

Two one-sided tests were performed on DESeq2 $\log_2$ fold-change estimates using a predefined margin $\delta = 0.75$. Genes were considered conserved if:

$$\texttt{padj\_equiv} < 0.05 \quad \text{and} \quad |\log_2 FC| < \delta$$

This distinguishes true similarity from non-significance.

12.6 Pathway enrichment analysis

Pathway analyses were performed using MSigDB Hallmark and C7 collections.

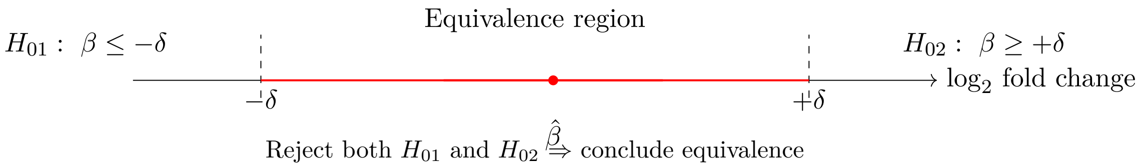


Figure 1: Two one-sided tests (TOST) for equivalence of $\log_2$ fold change within $[-\delta, +\delta]$.

- GSEA (fgsea) on ranked gene lists (ranked by $\log_2$FC)
- ORA (clusterProfiler) on marker and conserved gene sets

Cross-contrast intersections and union gene sets were also tested for enrichment.

12.7 Cell-type–specific systems-level inference and consensus network modeling

Gene co-expression networks were constructed separately for each donor within predefined cell-type subsets (`CD4 T cells, B cells, and CD14 monocytes`). For each donor and cell-type subset, gene–gene association networks were inferred from metacell expression profiles using the **Muumi** framework, which implements mutual-information–based network inference through the `minet` package [10].

Metacell aggregation To stabilize gene–gene association estimates, cells were randomly aggregated into metacells (size = 20; seed = 42). Expression values were aggregated in linear scale (`expm1`) and log-transformed after pooling (`log1p`). Metacell expression was computed as the mean expression across pooled cells.

Gene filtering Within each donor × cell-type subset, genes detected (expression > 0) in at least 5% of metacells were retained. From these, up to 4000 highly variable genes (HVGs) were selected using Seurat's `FindVariableFeatures`.

Network inference (Muumi) Gene–gene association networks were inferred using the Muumi consensus framework. The `CLR` (Context Likelihood of Relatedness) algorithm was used with a Spearman association estimator and no discretization. Muumi produces a ranked gene–gene adjacency matrix where lower ranks correspond to stronger inferred associations.

Per-donor network sparsification Ranked associations were converted to edge weights using

$$w_{ij} = \frac{1}{\mathrm{rank}_{ij}}$$

Edges were retained using a per-gene sparsification strategy retaining the top-k neighbors per gene ($k = 75$). Resulting donor networks were simplified to undirected graphs and restricted to their largest connected component.

Cross-donor consensus construction The consensus approach ensures that the resulting network topology represents emergent biological properties consistent across the entire study population.

Donor-specific networks were integrated using a replicate-stability consensus strategy. For each gene pair we computed:

- `support`: number of donors in which the edge was observed
- `median_weight`: median Muumi edge weight across supporting donors

Edges were retained if observed in at least two donors ($n = 4$ donors total). The resulting consensus network was restricted to its largest connected component.

Module detection Network modules were identified using the Louvain community detection algorithm implemented in the Muumi framework.

Consensus edge weighting For visualization and prioritization, a stability-adjusted score was computed:

$$\texttt{weight\_consensus} = \texttt{median\_weight} \times \texttt{support\_frac}, \qquad \texttt{support\_frac} = \frac{\texttt{support}}{\#\text{ donors}}$$

For visualization in Gephi, this value was transformed to a positive log-scale edge weight:

$$\texttt{weight\_consensus\_log} = \max(-\log_{10}(\texttt{weight\_consensus})) - (-\log_{10}(\texttt{weight\_consensus})) + 1$$

This transformation improves dynamic range while preserving relative edge strengths.

Functional enrichment Module gene sets were tested for over-representation using MSigDB Hallmark (H) and Immunologic Signatures (C7) gene sets, as well as Reactome pathways.

12.8 Network exports for Gephi

For each consensus network, Gephi-compatible tables were generated:

- Edge table (`edges.csv`) including `weight_raw`, `support`, `support_frac`, and `weight_consensus`
- Node table (`nodes.csv`) including module assignment, degree, and betweenness centrality
- Module membership table (`modules.csv`)

Nodes are additionally annotated with canonical marker membership flags and DEG-derived indicators to facilitate biological interpretation.

12.9 Network visualization with Gephi

Consensus networks were visualized using Gephi v0.10.1. Gephi was used only for layout and graphical rendering; all network construction and analysis were performed within the reproducible R pipeline.

Layout algorithm Node positions were computed using the ForceAtlas2 layout algorithm with the following parameters:

- Scaling = 200
- Gravity = 0.2
- Edge weight influence = 1.0
- Dissuade hubs = enabled
- Prevent overlap = enabled

Layouts were allowed to converge visually before export.

Graph filtering Visualization was restricted to the giant component. Edges were additionally filtered using an approximate weight threshold of ~ 1.2 to remove weak associations and improve readability.

Visual encoding

- Node color: Louvain module assignment
- Node size: Degree centrality
- Edge thickness: log-transformed consensus edge weight (`weight_consensus_log`)

Export Final layouts were exported as vector graphics (PDF/SVG) for inclusion in the report.

13 Results

13.1 Quality control and filtering outcomes

Donor	Cells	Med. nFeat	Med. nCount	Med. %mt	%mt>20	Removed
Donor1	5510	2058	7889	15.77	27.33	1506
Donor2	5525	2127	7932	14.61	21.38	1181
Donor3	4612	2062	8608	15.72	23.24	1072
Donor4	5206	1786	6819	23.68	63.81	3322

Table 5: Pre-filtering quality control metrics per donor (mitochondrial threshold = 20%).

Initial cell recovery ranged from 4,612 to 5,525 cells per donor (Table 5). Median detected features per cell were consistent across donors (1,786–2,127 genes), with donor4 exhibiting slightly

Donor	Median %ribo	Median %hb
Donor1	21.24	0.00
Donor2	15.12	0.00
Donor3	21.01	0.00
Donor4	11.84	0.00

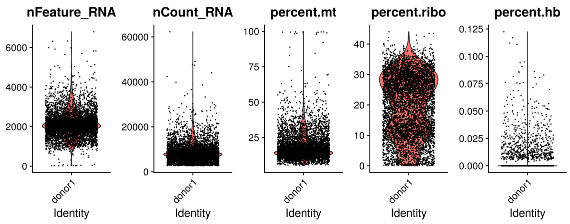
Table 6: Additional QC composition metrics per donor.

lower complexity. Median UMI counts ranged from 6,819 to 8,608.

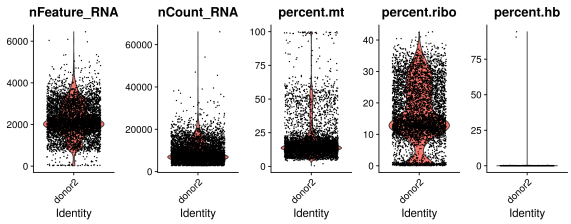
Mitochondrial content differed substantially between donors. Donors 1–3 showed comparable median mitochondrial percentages (∼14–16%), whereas donor4 displayed elevated mitochondrial content (median 23.7%). Consequently, 63.8% of donor4 cells exceeded the 20% mitochondrial threshold compared to 21–27% in the other donors (Table 5).

Feature-count extremes were rare: fewer than 1.5% of cells exhibited < 500 detected genes, and $< 0.2\%$ exceeded 6,000 genes in any donor, indicating minimal evidence of low-quality empty droplets or high-confidence doublets based on feature count alone.

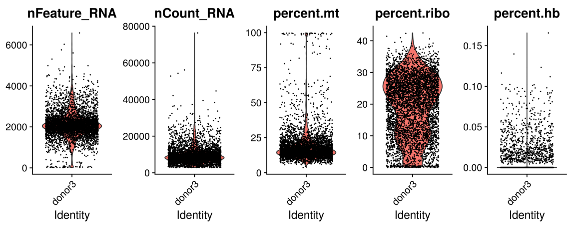
Overall, quality distributions were broadly consistent across donors with the exception of donor4, which showed a right-shifted mitochondrial distribution and lower median transcript complexity (Figure 2).



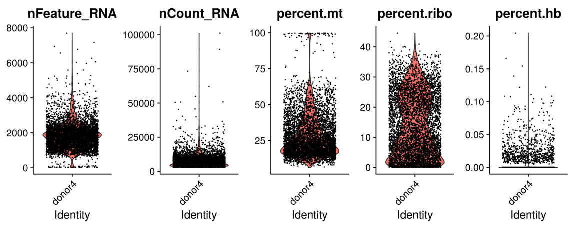
(a) Donor1



(b) Donor2



(c) Donor3



(d) Donor4

Figure 2: Quality control distributions per donor showing nFeature_RNA, nCount_RNA, percent.mt, percent.ribo and percent.hb prior to filtering.

13.2 Filtering yield and failure modes

Quality control was performed independently for each donor using donor-specific empirical thresholds.

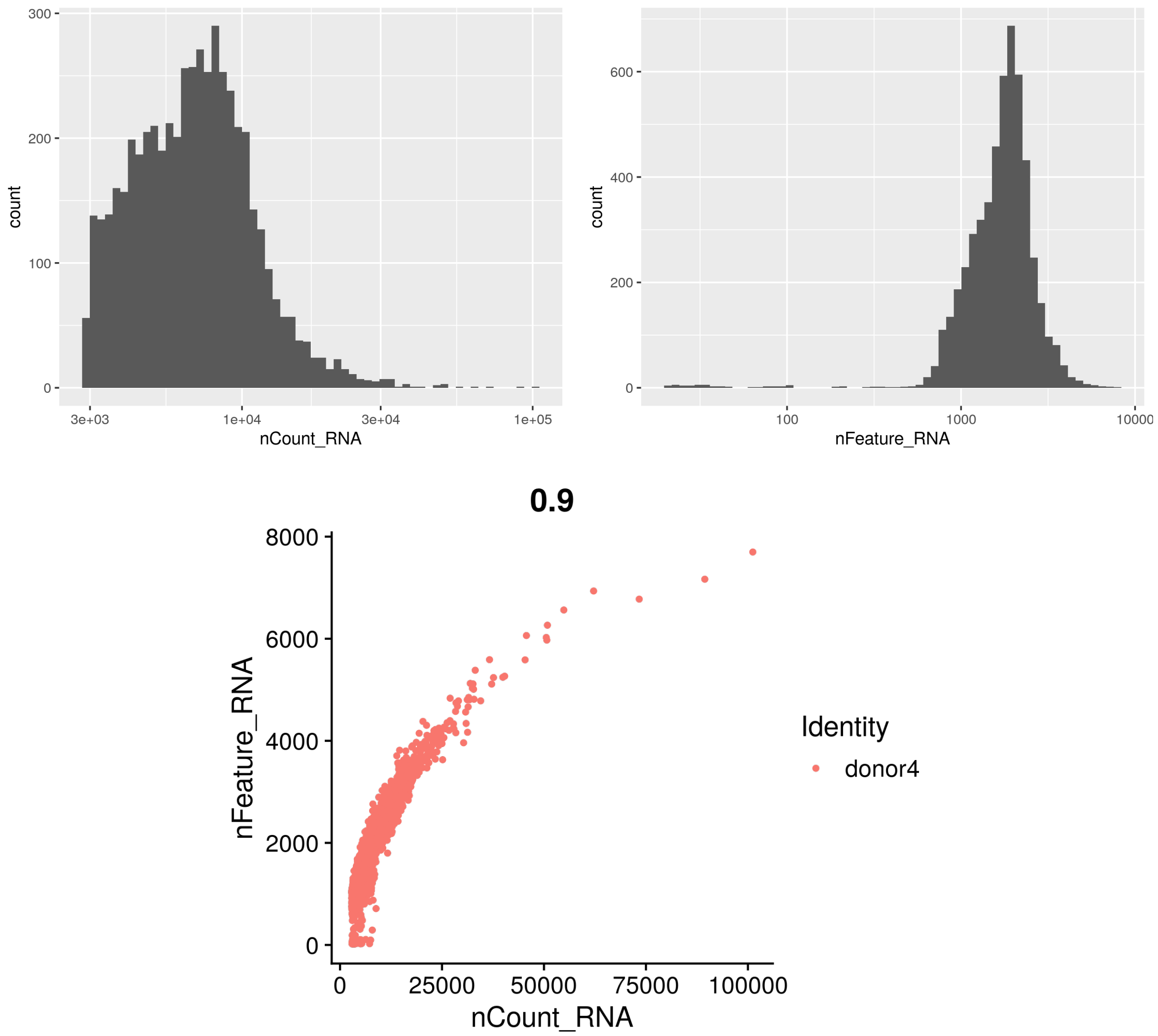


Figure 3: Quality control metrics for donor4 prior to filtering. Histograms show UMI count and detected gene distributions; scatter plot shows the relationship between UMI counts and detected genes per cell.

Across donors, 14–18% of cells were removed for Donors 1–3, whereas donor4 exhibited substantially higher mitochondrial content, resulting in removal of 47.6% of cells (Table 7).

Mitochondrial percentage was the dominant filtering driver across donors (Tables 7–8).

Representative pre- and post-filtering distributions for donor4 are shown in Figures 3 and 4. Filtering reduced high-mitochondrial outliers and extreme library size artifacts while preserving the main cellular population.

13.3 Clustering structure and annotation consistency

UMAP embeddings revealed well-resolved immune lineages across all donors (Figure 5). Major PBMC populations—including CD4_T, CD8_T, NK, B-cells, CD14 monocytes, and FCGR3A

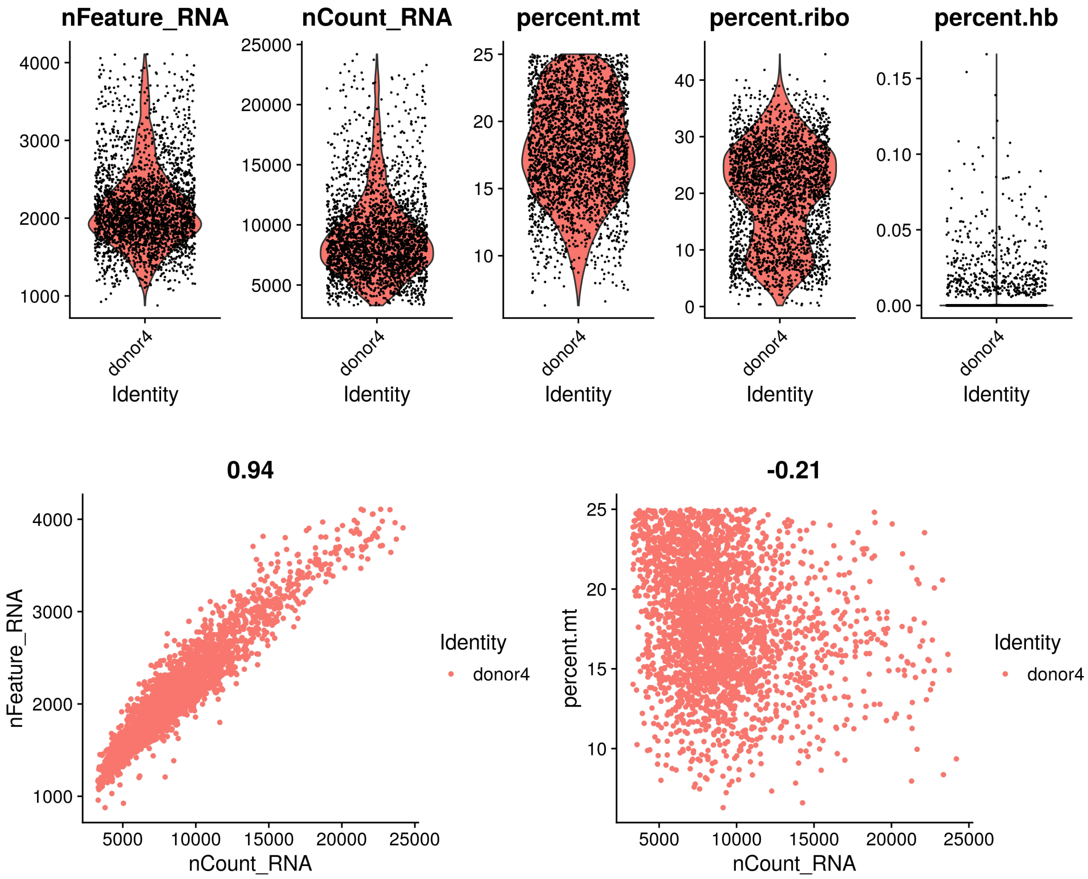


Figure 4: Quality control metrics for donor4 after quality filtering and normalization. Top: distribution of detected features, UMI counts, and mitochondrial percentage. Bottom: relationships between UMI counts and features (left) and mitochondrial content (right).

monocytes—formed discrete and reproducible clusters.

CD4_T cells represented the largest population in all donors (1,316–3,003 cells), followed by CD14_Mono and B-cells (Table 9). CD8_T and NK cells formed smaller but clearly separable clusters. Minor populations including dendritic cells, plasma cells, and platelets were consistently detected at low frequency.

Cluster topology was conserved across donors. T cell subsets localized to adjacent regions, monocyte subsets formed distinct but proximal clusters, and B-lineage cells separated clearly from myeloid populations. No major donor-specific distortions of global manifold structure were observed.

Low-count generic "T_cells" annotations were minimal (0–7 cells per donor), indicating strong agreement between per-cell predictions and majority cluster identities. Overall, clustering and annotation were consistent across donors, supporting downstream pseudobulk.

Donor	Cells (before)	Cells (after)	% dropped	MT fails
Donor1	5510	4499	18.35	885
Donor2	5525	4618	16.42	808
Donor3	4612	3951	14.33	541
Donor4	5206	2729	47.58	2400

Table 7: Cell filtering summary per donor.

Donor	Cells (pre)	Cells removed	Cells retained	% removed
Donor1	5510	1506	4004	27.3
Donor2	5525	1181	4344	21.4
Donor3	4612	1072	3540	23.2
Donor4	5206	3322	1884	63.8

Table 8: Cells removed based on mitochondrial content threshold ($> 20\%$).

13.4 Differential expression between coarse immune groups

Pseudobulk differential expression was performed across three coarse immune contrasts with balanced donor support (8 pseudobulk samples derived from 4 paired donors per contrast; Table 10).

Strict marker genes were most abundant for T-like vs Mono-like (1,366 genes), followed by B-like vs Mono-like (674) and T-like vs B-like (464). In contrast, equivalence testing identified the largest conserved set for T-like vs B-like (2,266 genes), with substantially fewer conserved genes in contrasts involving monocyte populations (641 and 199 genes, respectively).

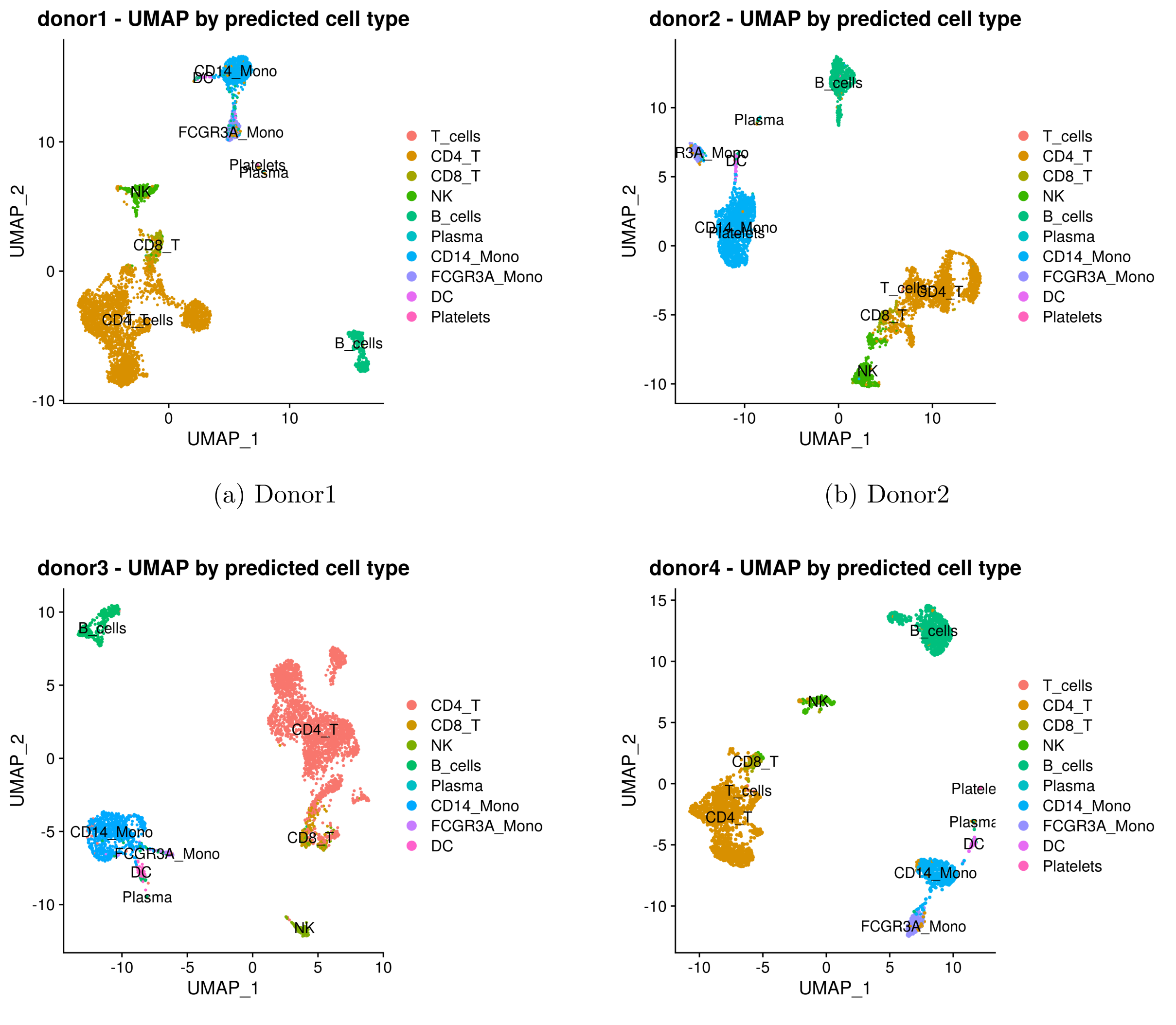
Volcano plots show clear separation between large-effect marker genes and small-effect conserved genes across contrasts (Figure 6). Effect-size distributions demonstrate that most genes concentrate near zero fold change, with strict markers occupying the distribution tails and equivalence calls localized near the origin (Figure 7).

Together, these results indicate strong transcriptional divergence between lymphoid and myeloid compartments, while also identifying contrast-specific sets of genes with statistically supported minimal differences.

Cross-contrast aggregation highlights the structure of differential and equivalence signals (Table 11). A total of 1,763 genes were identified as strict markers in at least one contrast, whereas only 13 genes were shared as markers across all three contrasts, indicating strong contrast specificity of large-effect transcriptional differences.

In contrast, conserved (equivalence) genes were more stable across comparisons: 2,588 genes were conserved in at least one contrast and 92 were conserved across all three contrasts, suggesting the presence of a shared transcriptional core with statistically supported small effect sizes.

Overlap between marker and conserved sets was limited (20 genes), consistent with the mutually exclusive effect-size definitions used for classification.



(a) Donor1 (b) Donor2

(c) Donor3 (d) Donor4

Figure 5: UMAP embeddings per donor colored by predicted cell type. Major PBMC lineages form discrete, well-separated clusters across donors.

13.5 Pathway enrichment patterns

Pathway enrichment was evaluated per contrast using two complementary approaches: GSEA on ranked gene lists and ORA on discrete strict marker and conserved gene sets. For concise presentation, results are additionally summarized using aggregated ("any contrast") gene sets to illustrate shared pathway-level trends across comparisons.

Hallmark ORA on marker genes identified enrichment of pathways consistent with large, contrast-specific transcriptional differences (Figure 8). In contrast, ORA on conserved genes highlighted pathways over-represented among genes with statistically supported small effects, providing a complementary view of stable transcriptional programs across comparisons (Figure 8).

C7 ORA identified immune-context signatures and provided higher-resolution immunologic

Cell type (%)	Donor1	Donor2	Donor3	Donor4
CD4_T	66.8	38.3	62.1	48.2
CD8_T	2.8	1.9	3.5	2.9
T_cells	0.1	0.2	0.0	0.2
NK	6.0	9.6	4.1	5.3
B_cells	8.8	12.8	8.4	22.3
Plasma	0.1	0.3	0.2	0.3
CD14_Mono	12.2	32.8	19.1	14.3
FCGR3A_Mono	2.4	2.8	1.4	5.0
DC	0.9	1.3	1.2	1.2
Platelets	0.0	0.1	0.0	0.4

Table 9: Relative cell-type composition per donor (percent of post-filter cells).

Contrast	Samples	Donors	Strict markers	Conserved (TOST)
T-like vs B-like	8	4	464	2266
T-like vs Mono-like	8	4	1366	641
B-like vs Mono-like	8	4	674	199

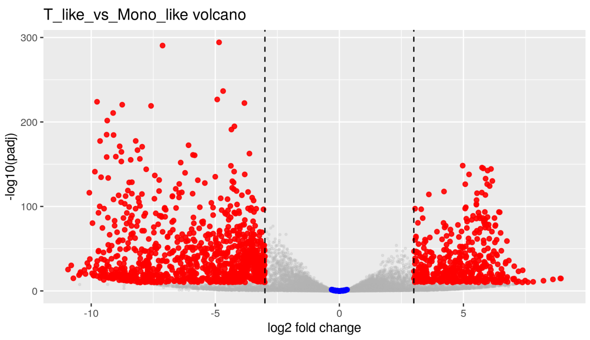
Table 10: Pseudobulk DESeq2 + TOST summary per contrast. Samples denote donor-by-group pseudobulks retained after donor pairing and minimum cell-count filtering. Strict markers use padj $< 10^{-10}$ and $|\log_2 \mathrm{FC}| > 3$. Conserved genes are equivalence calls (TOST) with FDR < 0.05 and $|\log_2 \mathrm{FC}| < \delta$ (with expression filtering by baseMean).

annotation of the marker and conserved sets (Figure 9). A representative Hallmark GSEA analysis illustrates concordant pathway-level directionality when using continuous ranked statistics (Figure 10).

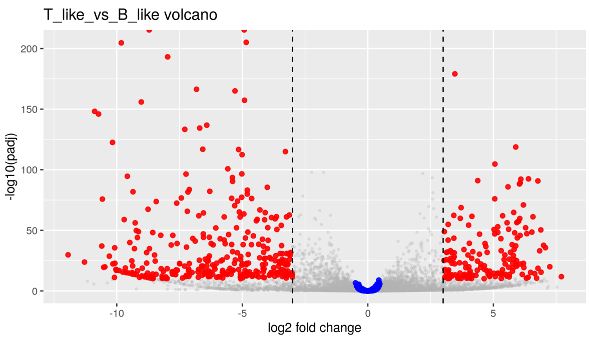
Hallmark over-representation analysis of aggregated strict marker genes was dominated by immune activation and inflammatory signaling pathways, including inflammatory response, allograft rejection, TNFA signaling via NFκB, complement, and cytokine signaling programs (IL2/STAT5 and IL6/JAK/STAT3). These enrichments are consistent with strong transcriptional divergence between lymphoid and myeloid compartments.

In contrast, conserved (equivalence) gene sets were enriched for MYC targets, oxidative phosphorylation, G2M checkpoint, unfolded protein response, protein secretion, and DNA repair. These pathways reflect shared metabolic, proliferative, and housekeeping programs across immune compartments.

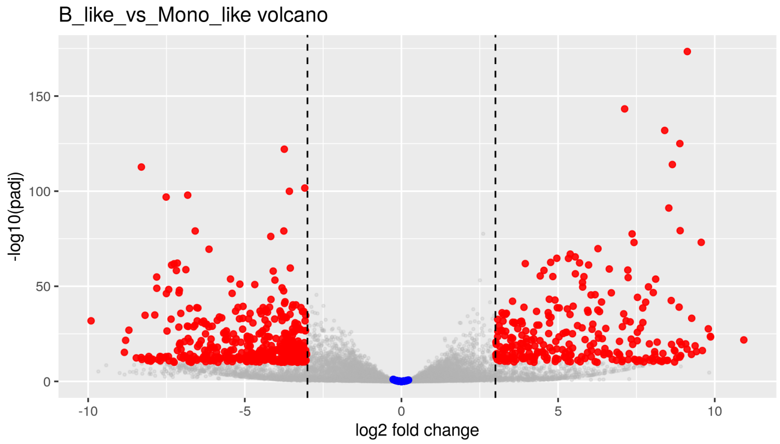
The minimal overlap in pathway-level enrichment between marker and conserved sets supports the effect-size-based separation of contrast-specific versus core transcriptional programs.



(a) T-like vs Mono-like



(b) T-like vs B-like



(c) B-like vs Mono-like

Figure 6: Volcano plots from pseudobulk DESeq2 contrasts. Red points indicate strict markers; blue points indicate genes passing equivalence (TOST). Dashed lines mark the strict marker log2FC threshold.

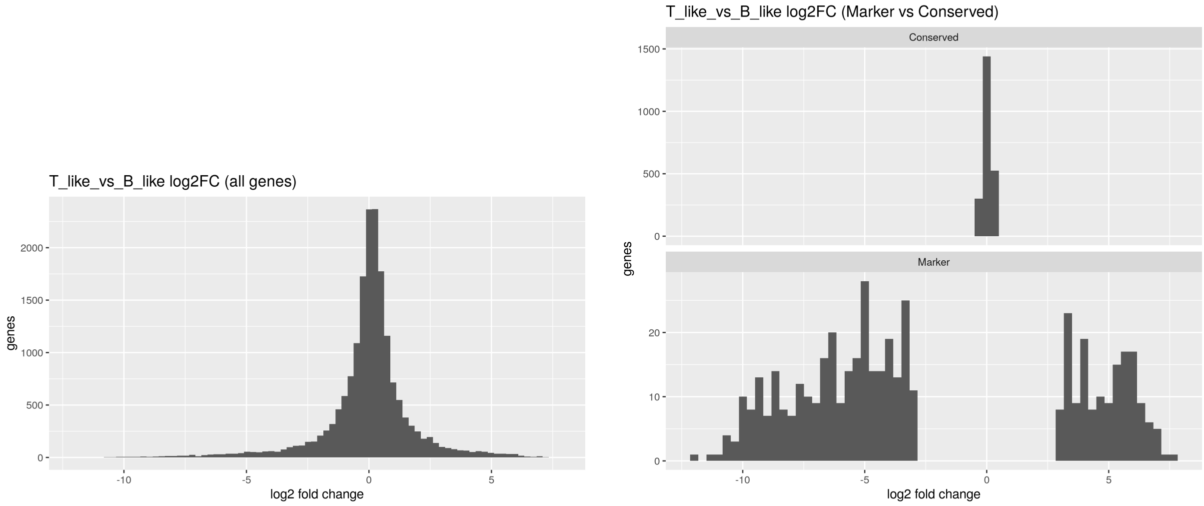
13.6 Consensus co-expression network properties

Consensus co-expression networks were constructed for CD4 T cells, B cells, and CD14 monocytes from donor-level **Muumi**-inferred gene association networks derived from metacell expression profiles. Donor networks were sparsified, integrated by cross-donor support filtering, and restricted to the largest connected component (LCC).

Final consensus networks were obtained after donor-level sparsification, cross-donor support filtering (≥ 2 donors), and restriction to the largest connected component. All three cell-type networks formed a single connected component post consensus-LCC (Table 13).

The support distributions in Figure 11 show that donor-specific edges dominate prior to consensus filtering, whereas the final retained edges are primarily supported in two donors, with progressively fewer edges supported in three or four donors.

Final consensus edge weights are summarized as Muumi-derived weights on a $-\log_{10}$ scale (Figure 13). The CD4 network shows a broad weight distribution concentrated toward higher transformed weights, consistent with retention of stronger replicate-supported edges after consen-



(a) All genes

(b) Marker vs conserved subsets

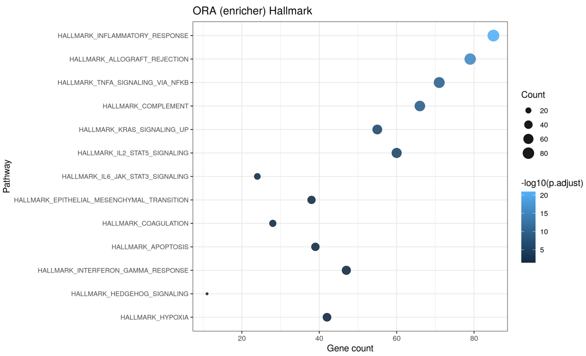
Figure 7: Distribution of DESeq2 log2 fold-changes for a representative contrast (T-like vs B-like). Most genes concentrate near 0, while strict marker calls occupy the tails; equivalence calls concentrate near 0 by construction.

Set	**Genes**
Markers (any contrast)	1763
Markers (all contrasts)	13
Conserved (any contrast)	2588
Conserved (all contrasts)	92
Marker ∩ Conserved	20

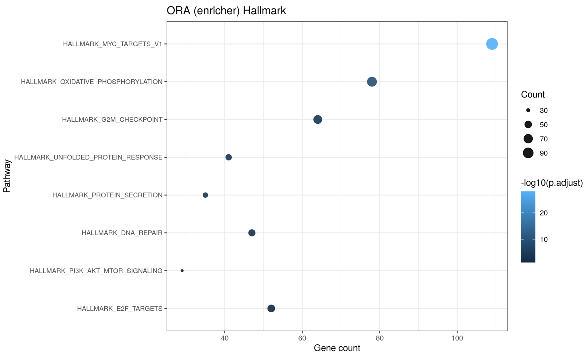
Table 11: Cross-contrast aggregation of strict marker and conserved (equivalent) gene sets.

sus filtering and LCC restriction.

Module size distributions are heterogeneous across all three networks (Figure 12), with a small number of large Louvain modules and a longer tail of smaller modules. CD4 T cells yielded 32 modules, whereas B cells and CD14 monocytes each yielded 50 modules (Table 13).

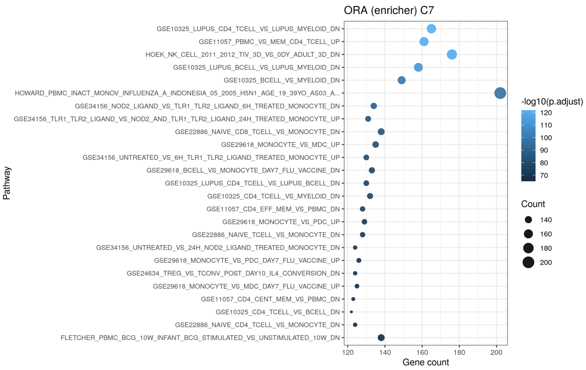


(a) Markers (any contrast)

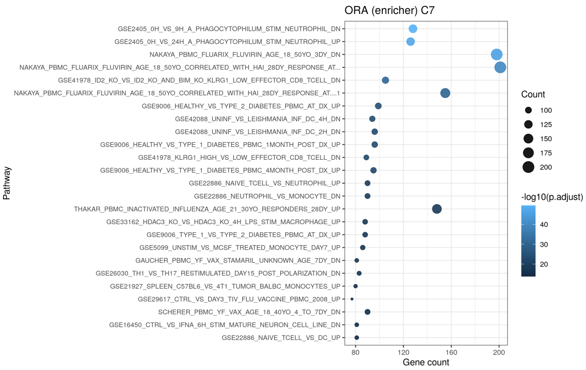


(b) Conserved (any contrast)

Figure 8: Over-representation analysis (Hallmark) for aggregated gene sets across contrasts.



(a) Markers (any contrast)



(b) Conserved (any contrast)

Figure 9: Over-representation analysis (C7) for aggregated gene sets across contrasts.

13.7 Module-level functional enrichment and network visualization

Consensus networks for CD4 T cells, B cells, and CD14 monocytes were visualized in Gephi after consensus construction and restriction to the largest connected component (Figures 14, 17, and 20). Nodes represent genes and edges represent consensus-supported co-expression relationships. Nodes are colored by Louvain module assignment and sized by degree centrality. Functional interpretation of modules was performed using over-representation analysis (ORA) against MSigDB Hallmark and C7 immunologic signature gene sets, complemented by Reactome pathway enrichment (Figures 15–22). Enrichment results were evaluated at the module level using the network gene universe as background.

CD4 T-cell network The CD4 network shows clear separation between adaptive immune signaling modules, interferon-associated programs, and a distinct translation-associated module (Figure 14). Hallmark enrichment identifies modules associated with TNFα/NFκB signaling, interferon responses, complement, allograft rejection, and MYC targets (Figure 15, top). C7 enrichment further supports this structure, with modules enriched for T-cell activation, vaccination-response, and infection-associated signatures (Figure 15, bottom). Reactome enrichment highlights canonical immune signaling processes including TCR signaling, CD28 co-stimulation, PD-1 signaling, interleukin signaling, and interferon signaling (Figure 16). Overall, the CD4 network captures coordinated adaptive immune activation together with a separate translational program.

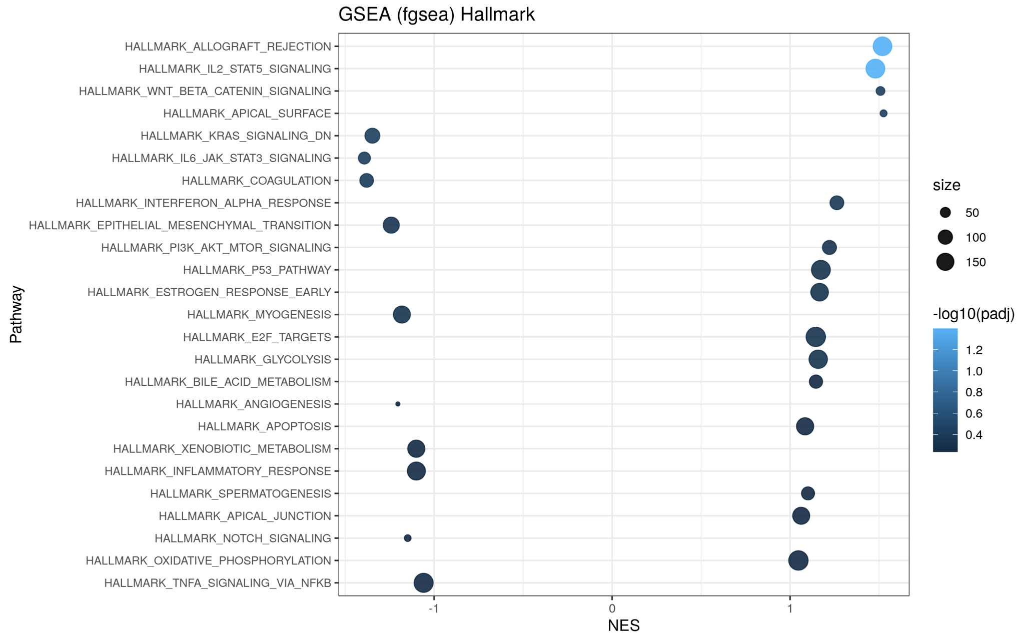


Figure 10: GSEA (Hallmark) for T-like vs B-like ranked by $\log_2$FC.

B-cell network The B-cell network is more compact, with enrichment concentrated in a smaller number of modules (Figure 17). Hallmark enrichment identifies modules associated with complement, inflammatory response, KRAS signaling, coagulation, and metabolic pathways (Figure 18, top). C7 enrichment highlights vaccine-response, infection-associated, and naive-versus-memory or activation-state contrasts (Figure 18, bottom). Reactome enrichment additionally identifies platelet activation, platelet degranulation, extracellular matrix interaction, and related pathways (Figure 19). Inspection of the network visualization shows a distinct platelet-associated module containing genes such as *PF4*, *PPBP*, *GP9*, *ITGB3*, and *TUBB1*, indicating a platelet-derived transcriptional signal within the B-cell subset. This pattern is interpreted cautiously, as it likely reflects residual platelet contamination or platelet–immune aggregates rather than intrinsic B-cell regulatory structure.

CD14 monocyte network The CD14-monocyte network displays the strongest innate immune signature of the three cell types (Figure 20). Hallmark enrichment identifies modules associated with TNFα/NFκB signaling, interferon alpha and gamma responses, allograft rejection, MYC targets, and hypoxia-related programs (Figure 21, top). C7 enrichment shows extensive overlap with LPS stimulation, viral infection, Toll-like receptor signaling, vaccine-response, and monocyte-versus-lymphocyte contrast datasets (Figure 21, bottom). Reactome enrichment similarly highlights interferon signaling, T-cell receptor-related signaling modules, cytokine signaling, and other immune activation processes (Figure 22). Compared with the adaptive networks, enrichment in the monocyte network is broader and more consistently aligned with pathogen-response biology.

Cross-network comparison Across all three networks, enrichment patterns support the biological coherence of the consensus modular structure. Interferon signaling and TNFα/NFκB-mediated inflammatory pathways recur across cell types but segregate into distinct modules

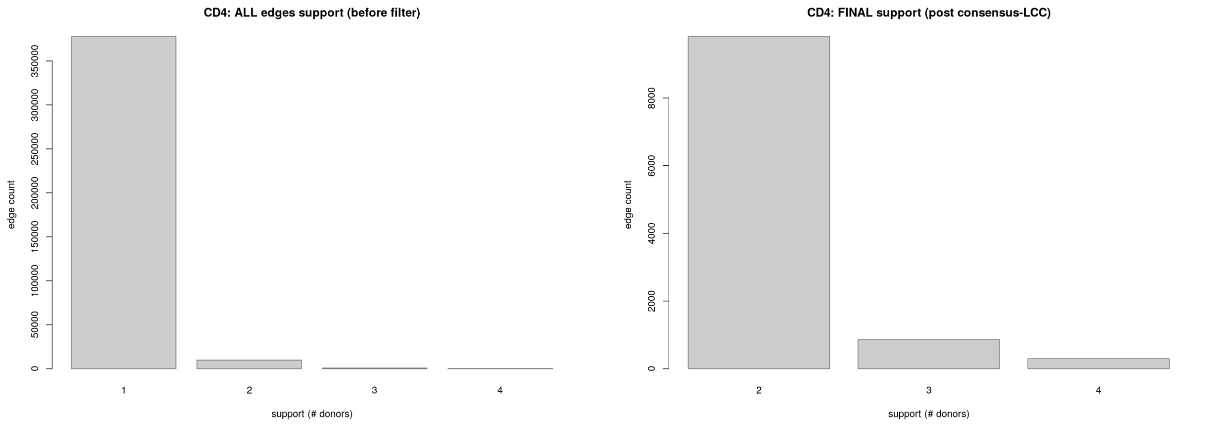
Gene set	Markers (any contrast)	Conserved (any contrast)
Immune activation	Inflammatory response	–
Antigen response	Allograft rejection	–
NFκB axis	TNFA signaling via NFκB	–
Innate immunity	Complement	–
Cytokine signaling	IL2/STAT5, IL6/JAK/STAT3	–
Proliferation	E2F targets	G2M checkpoint, E2F targets
Growth programs	KRAS signaling	MYC targets v1
Metabolism	–	Oxidative phosphorylation
Protein handling	–	Unfolded protein response, Protein secretion
Genome stability	–	DNA repair
mTOR axis	PI3K/AKT/mTOR	PI3K/AKT/mTOR

Table 12: Top enriched Hallmark pathways for aggregated strict marker genes and conserved (TOST) genes across contrasts. Markers are dominated by immune activation and inflammatory signaling programs, whereas conserved genes are enriched for shared metabolic, proliferative, and housekeeping pathways.

Network	Donors	Universe	Nodes	Edges	Comp	Modules
CD4 T cells	4	3948	3432	10972	1	32
B cells	4	4432	3272	6173	1	50
CD14 monocytes	4	4304	3268	6926	1	50

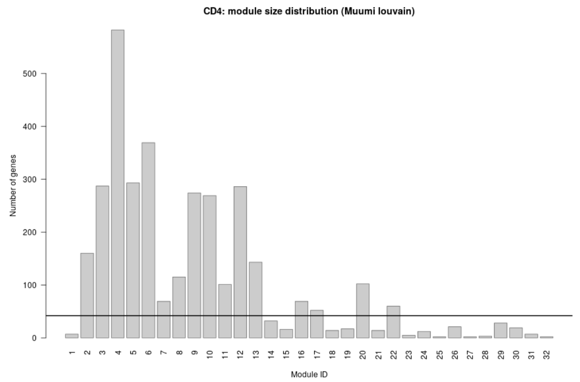
Table 13: Consensus Muumi network summary statistics after donor-level sparsification, cross-donor support filtering, and restriction to the largest connected component. Universe denotes genes eligible for consensus integration after donor-level filtering; Nodes and Edges refer to the final post-consensus, post-LCC graph; Comp = number of connected components; Modules = number of Louvain modules detected in the final graph.

within each network. The CD4 network emphasizes adaptive immune activation and translation-associated structure, the B-cell network captures activation and differentiation programs together with a detectable platelet-associated module, and the monocyte network is dominated by innate inflammatory and pathogen-response signatures. Given the modest size of some modules and the use of ORA, these interpretations should remain conservative; however, the enrichment structure is internally consistent and compatible with expected cell-type–specific immune biology.

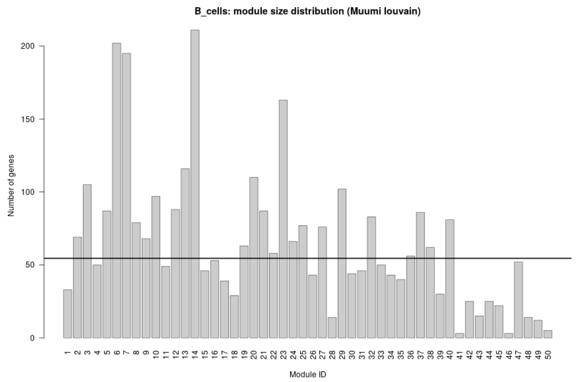


(a) All candidate edges (before support filtering) (b) Final consensus edges (post consensus + LCC)

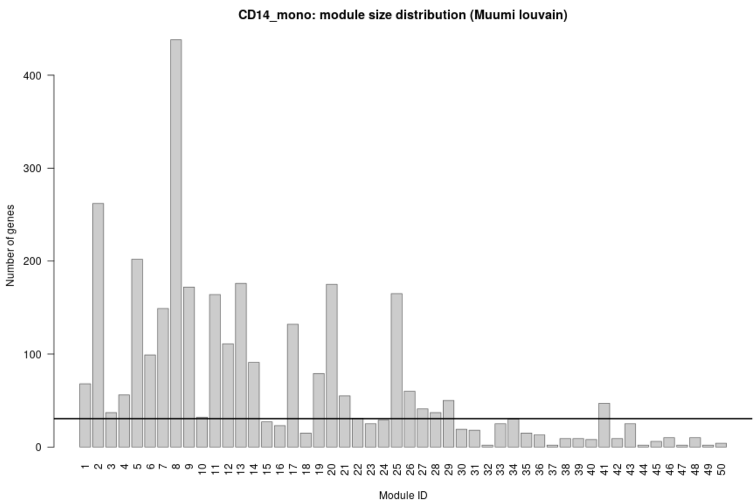
Figure 11: Edge support distributions for the CD4 T-cell network. Support denotes the number of donor-specific Muumi networks in which a gene–gene edge is observed. Before filtering, most candidate edges are donor-specific; after requiring support in at least two donors, the retained network is dominated by edges supported in exactly two donors, with smaller fractions supported in three or four donors.



(a) CD4 T cells



(b) B cells



(c) CD14 monocytes

Figure 12: Module size distributions for final post-consensus, post-LCC Muumi networks. Bars indicate the number of genes per Louvain module; the horizontal line denotes the median module size within each network.

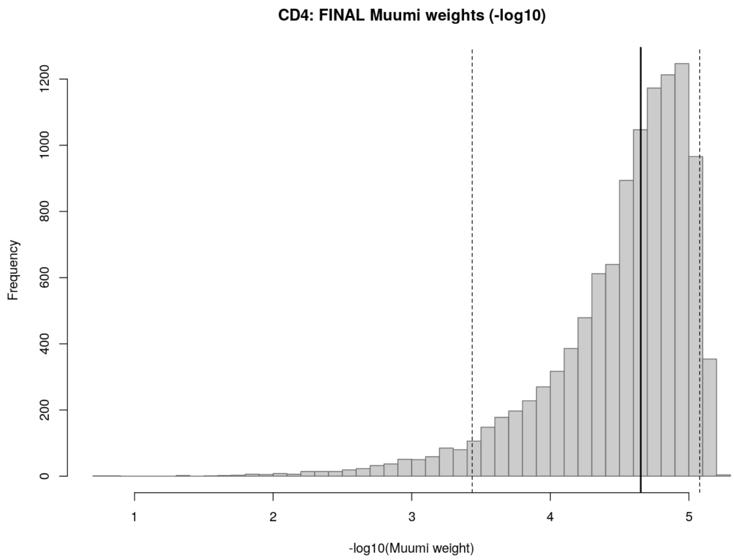


Figure 13: Distribution of final consensus Muumi edge weights on the $-\log_{10}$ scale after consensus filtering and LCC restriction (CD4 T cells shown). The solid vertical line marks the median; dashed lines indicate the 5th and 95th percentiles.

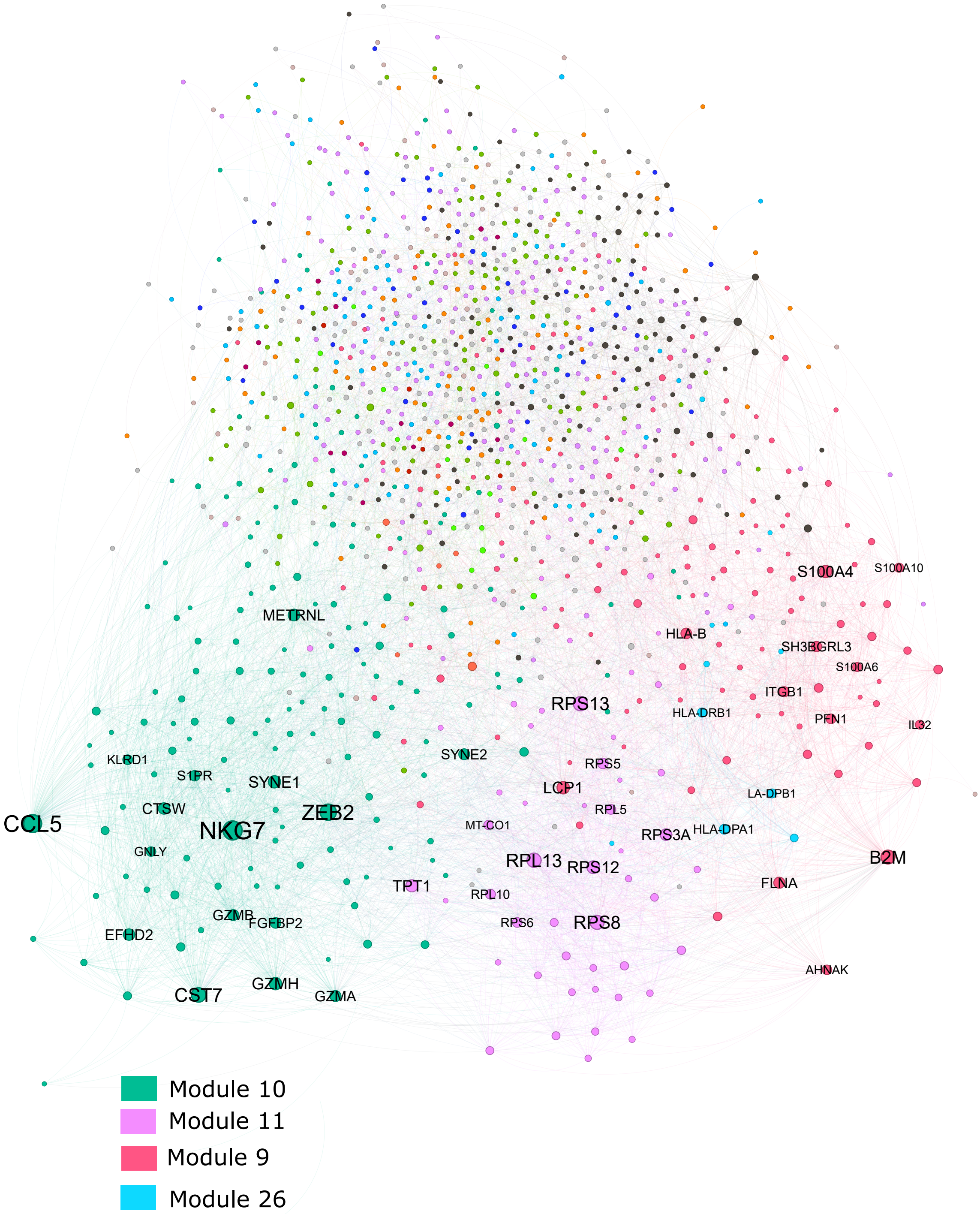
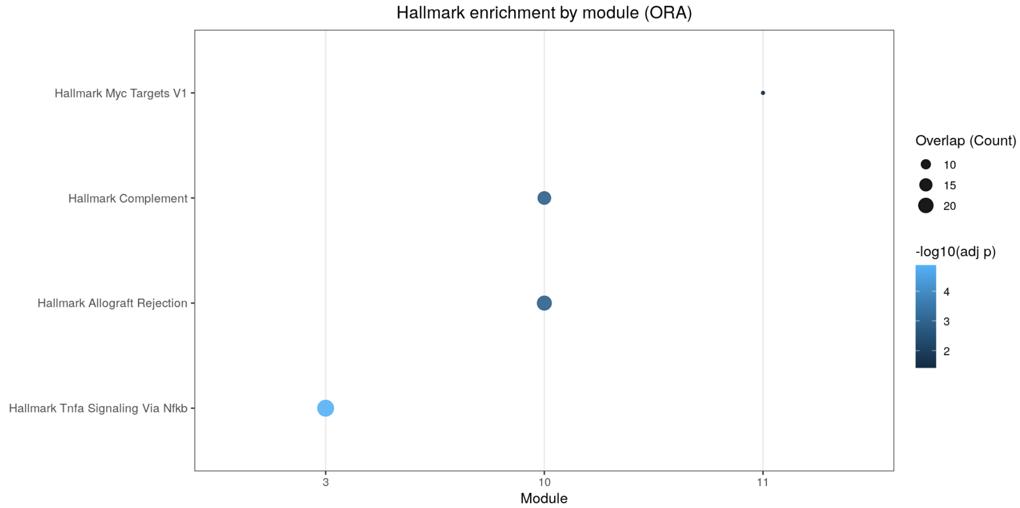
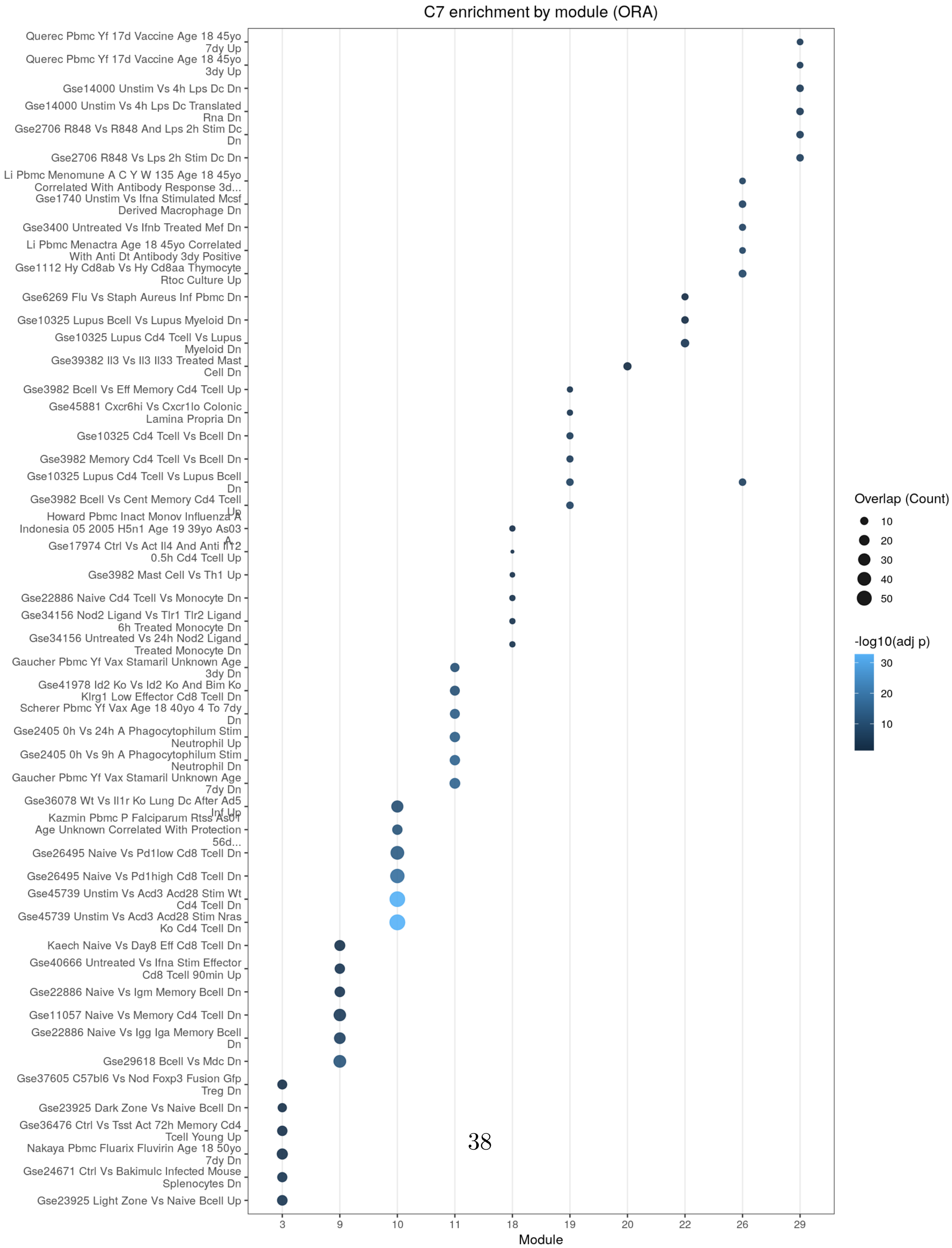


Figure 14: CD4 T-cell consensus network (post-consensus, post-LCC) colored by Louvain module.


Hallmark enrichment by module (ORA)
Hallmark Myc Targets V1
Hallmark Complement
Hallmark Allograft Rejection
Hallmark Tnfa Signaling Via Nfkb
3
10
11
Module
Overlap (Count)
10
15
20
-log10(adj p)
4
3
2


C7 enrichment by module (ORA)
Querec Pbmc Yf 17d Vaccine Age 18 45yo 7dy Up
Querec Pbmc Yf 17d Vaccine Age 18 45yo 3dy Up
Gse14000 Unstim Vs 4h Lps Dc Dn
Gse14000 Unstim Vs 4h Lps Dc Translated Rna Dn
Gse2706 R848 Vs R848 And Lps 2h Stim Dc Dn
Gse2706 R848 Vs Lps 2h Stim Dc Dn
Li Pbmc Menomune A C Y W 135 Age 18 45yo Correlated With Antibody Response 3d...
Gse1740 Unstim Vs Ifna Stimulated Mcsf Derived Macrophage Dn
Gse3400 Untreated Vs Ifnb Treated Mef Dn
Li Pbmc Menactra Age 18 45yo Correlated With Anti Dt Antibody 3dy Positive
Gse1112 Hy Cd8ab Vs Hy Cd8aa Thymocyte Rtoc Culture Up
Gse6269 Flu Vs Staph Aureus Inf Pbmc Dn
Gse10325 Lupus Bcell Vs Lupus Myeloid Dn
Gse10325 Lupus Cd4 Tcell Vs Lupus Myeloid Dn
Gse39382 Il3 Vs Il33 Treated Mast Cell Dn
Gse3982 Bcell Vs Eff Memory Cd4 Tcell Up
Gse45881 Cxcr6hi Vs Cxcr1lo Colonic Lamina Propria Dn
Gse10325 Cd4 Tcell Vs Bcell Dn
Gse3982 Memory Cd4 Tcell Vs Bcell Dn
Gse10325 Lupus Cd4 Tcell Vs Lupus Bcell Dn
Gse3982 Bcell Vs Cent Memory Cd4 Tcell Up
Howard Pbmc Inact Monov Influenza A Indonesia 05 2005 H5n1 Age 19 39yo As03 A
Gse17974 Ctrl Vs Act Il4 And Anti Il12 0.5h Cd4 Tcell Up
Gse3982 Mast Cell Vs Th1 Up
Gse22886 Naive Cd4 Tcell Vs Monocyte Dn
Gse34156 Nod2 Ligand Vs Tlr1 Tlr2 Ligand 6h Treated Monocyte Dn
Gse34156 Untreated Vs 24h Nod2 Ligand Treated Monocyte Dn
Gaucher Pbmc Yf Vax Stamaril Unknown Age 3dy Dn
Gse41978 Id2 Ko Vs Id2 Ko And Bim Ko Klrg1 Low Effector Cd8 Tcell Dn
Scherer Pbmc Yf Vax Age 18 40yo 4 To 7dy Dn
Gse2405 0h Vs 24h A Phagocytophilum Stim Neutrophil Up
Gse2405 0h Vs 9h A Phagocytophilum Stim Neutrophil Dn
Gaucher Pbmc Yf Vax Stamaril Unknown Age 7dy Dn
Gse36078 Wt Vs Il1r Ko Lung Dc After Ad5 Inf Up
Kazmin Pbmc P Falciparum Rtss As01 Age Unknown Correlated With Protection 56d...
Gse26495 Naive Vs Pd1low Cd8 Tcell Dn
Gse26495 Naive Vs Pd1high Cd8 Tcell Dn
Gse45739 Unstim Vs Acd3 Acd28 Stim Wt Cd4 Tcell Dn
Gse45739 Unstim Vs Acd3 Acd28 Stim Nras Ko Cd4 Tcell Dn
Kaech Naive Vs Day8 Eff Cd8 Tcell Dn
Gse40666 Untreated Vs Ifna Stim Effector Cd8 Tcell 90min Up
Gse22886 Naive Vs Igm Memory Bcell Dn
Gse11057 Naive Vs Memory Cd4 Tcell Dn
Gse22886 Naive Vs Igg Iga Memory Bcell Dn
Gse29618 Bcell Vs Mdc Dn
Gse37605 C57bl6 Vs Nod Foxp3 Fusion Gfp Treg Dn
Gse23925 Dark Zone Vs Naive Bcell Dn
Gse36476 Ctrl Vs Tsst Act 72h Memory Cd4 Tcell Young Up
Nakaya Pbmc Fluarix Fluvirin Age 18 50yo 7dy Dn
Gse24671 Ctrl Vs Bakimulc Infected Mouse Splenocytes Dn
Gse23925 Light Zone Vs Naive Bcell Up
3
9
10
11
18
19
20
22
26
29
Module
Overlap (Count)
10
20
30
40
50
-log10(adj p)
30
20
10

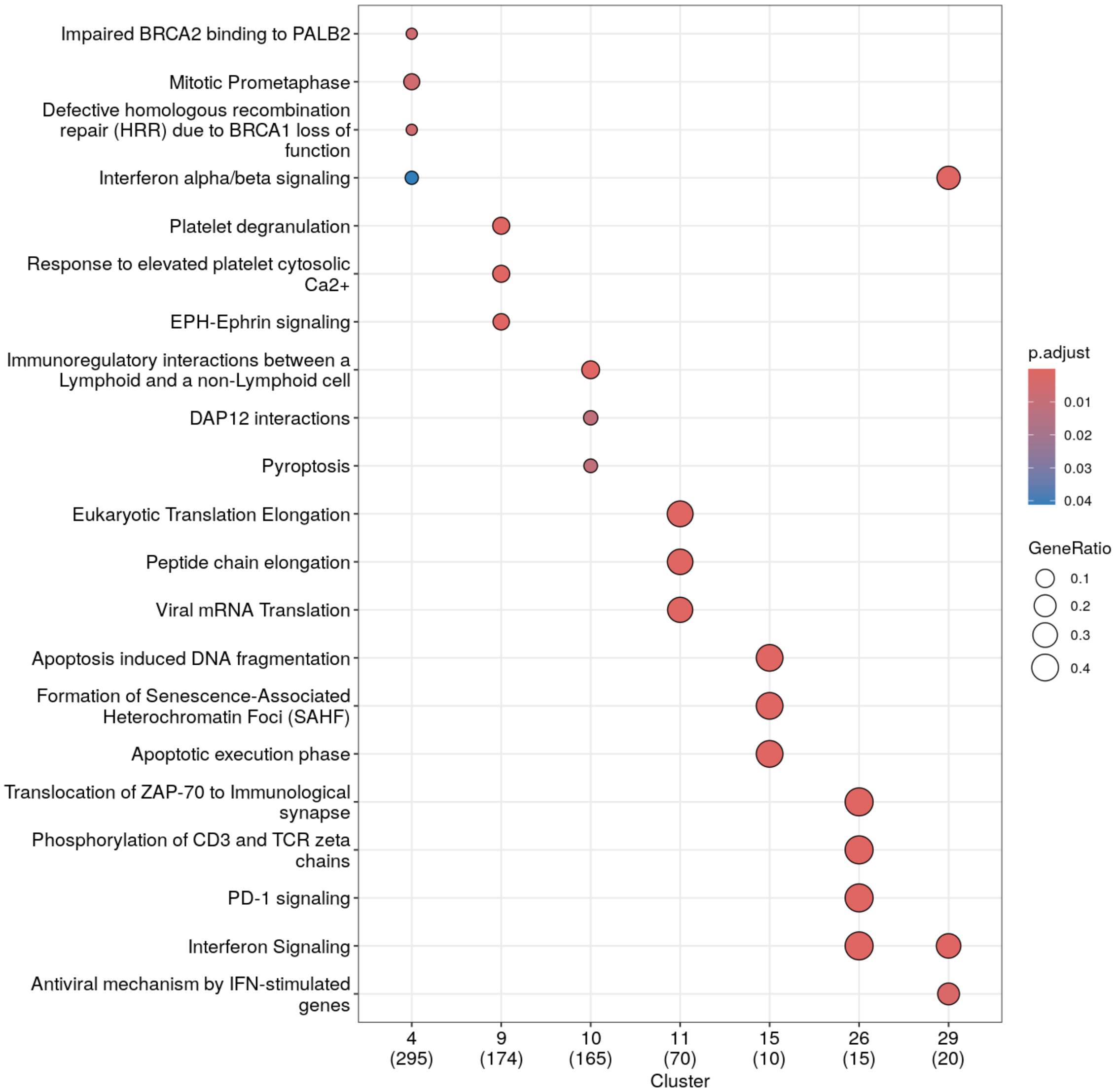


Figure 16: Reactome enrichment for modules in the CD4 T-cell consensus network.

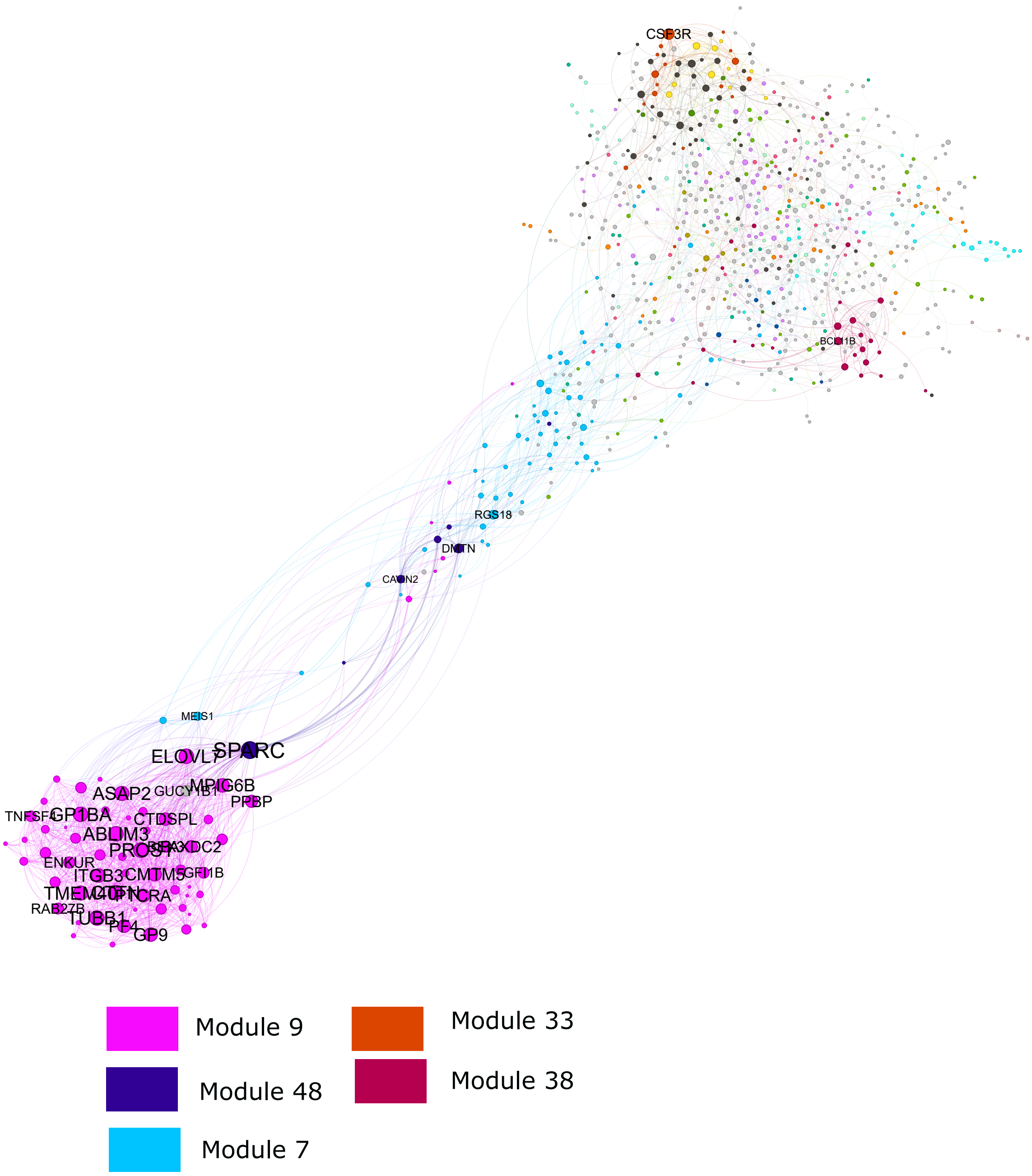


Figure 17: B-cell consensus network (post-consensus, post-LCC) colored by Louvain module.

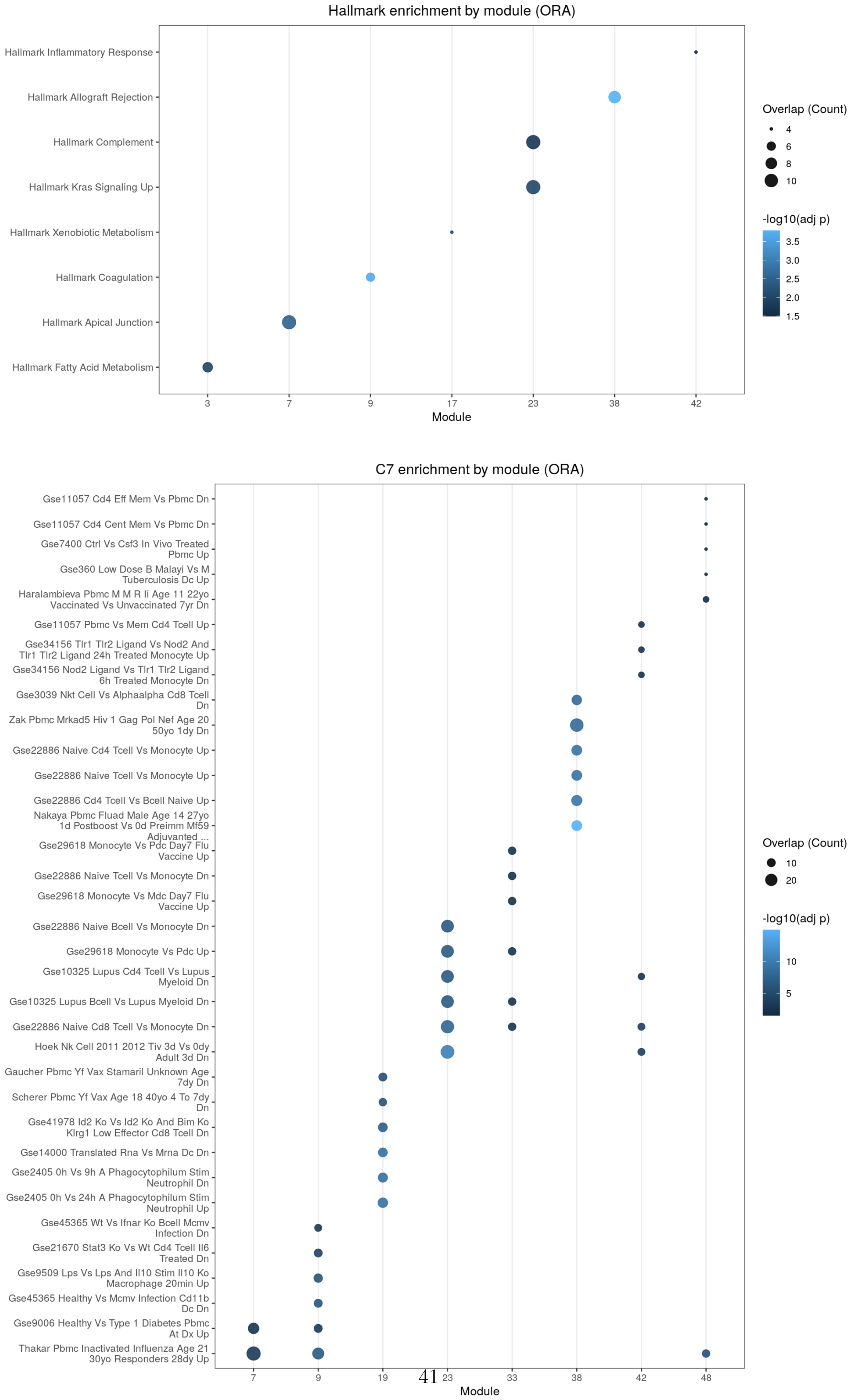


Figure 18: Module-level Hallmark and C7 enrichment for the B-cell consensus network. Top: Hallmark ORA. Bottom: C7 ORA.

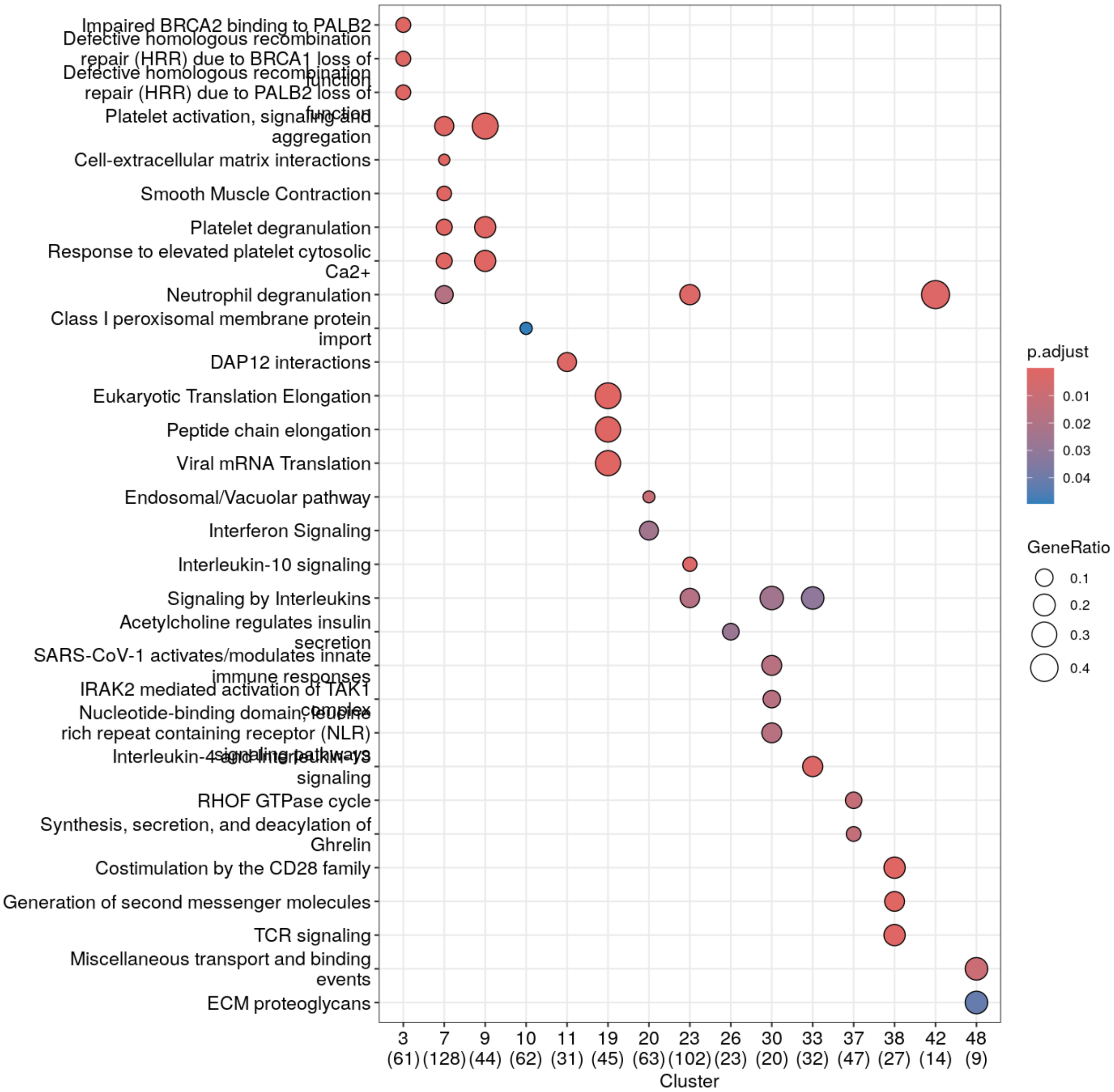


Figure 19: Reactome enrichment for modules in the B-cell consensus network.

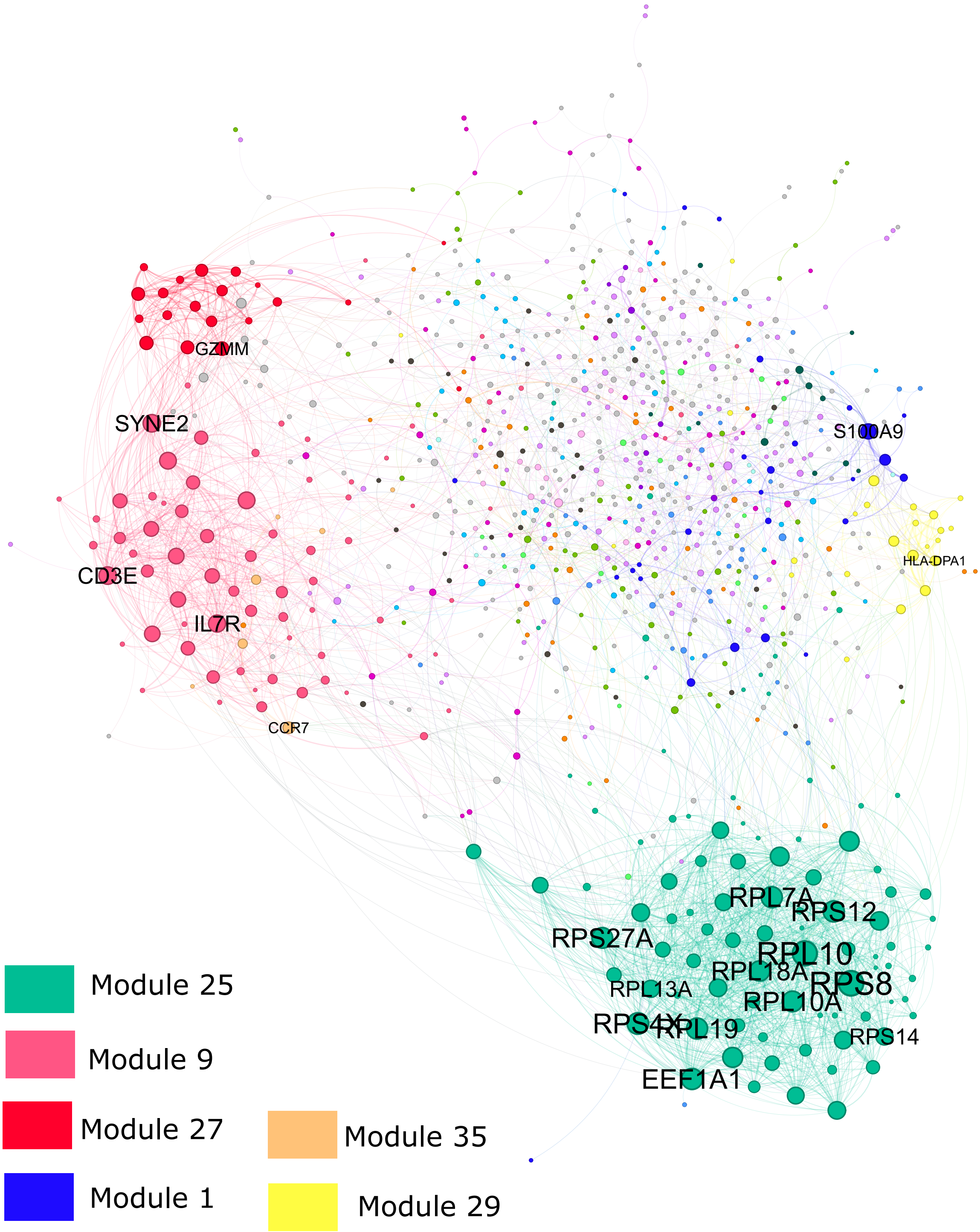


Figure 20: CD14-monocyte consensus network (post-consensus, post-LCC) colored by Louvain module.

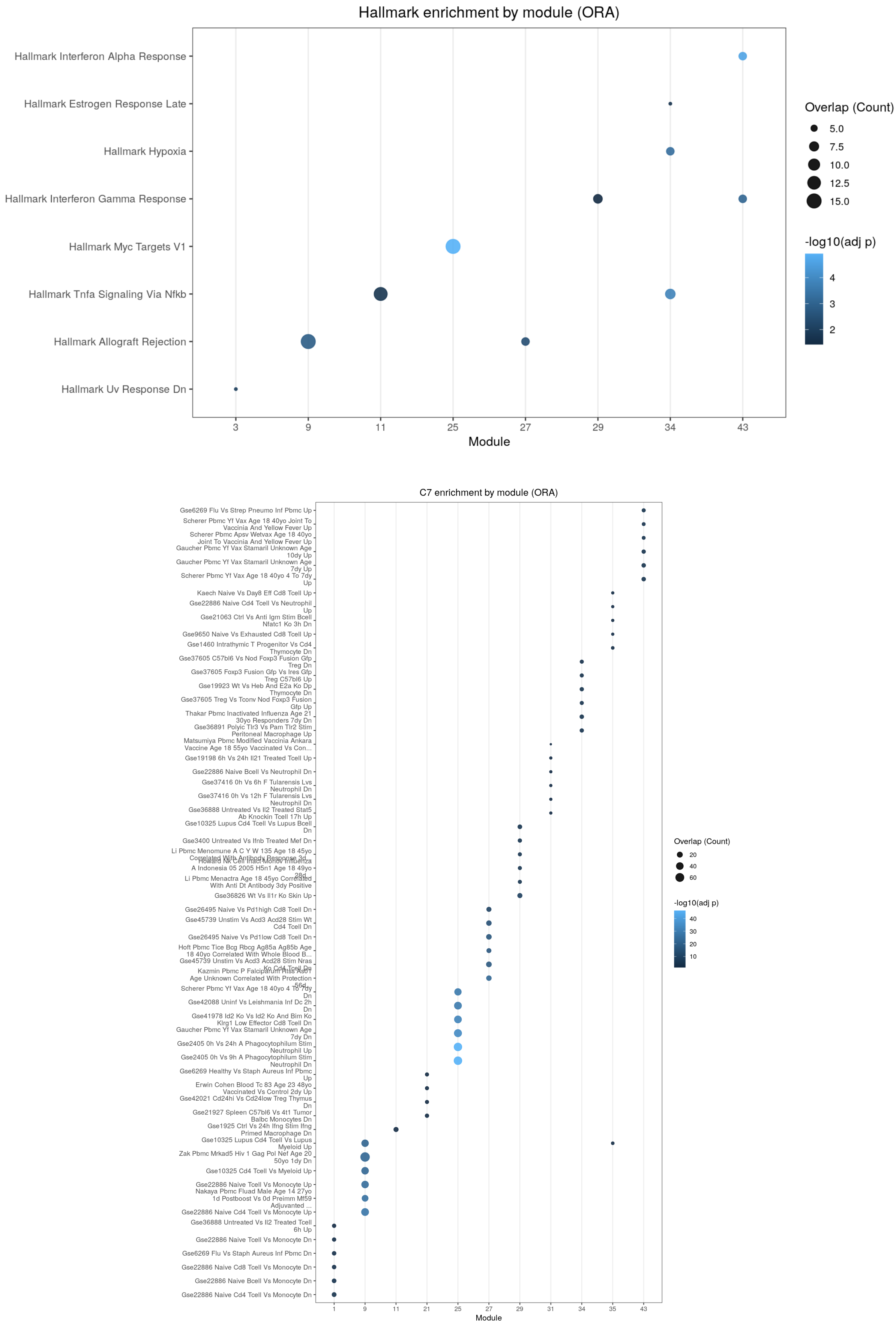


Figure 21: Module-level Hallmark and C7 enrichment for the CD14-monocyte consensus network. Top: Hallmark ORA. Bottom: C7 ORA.

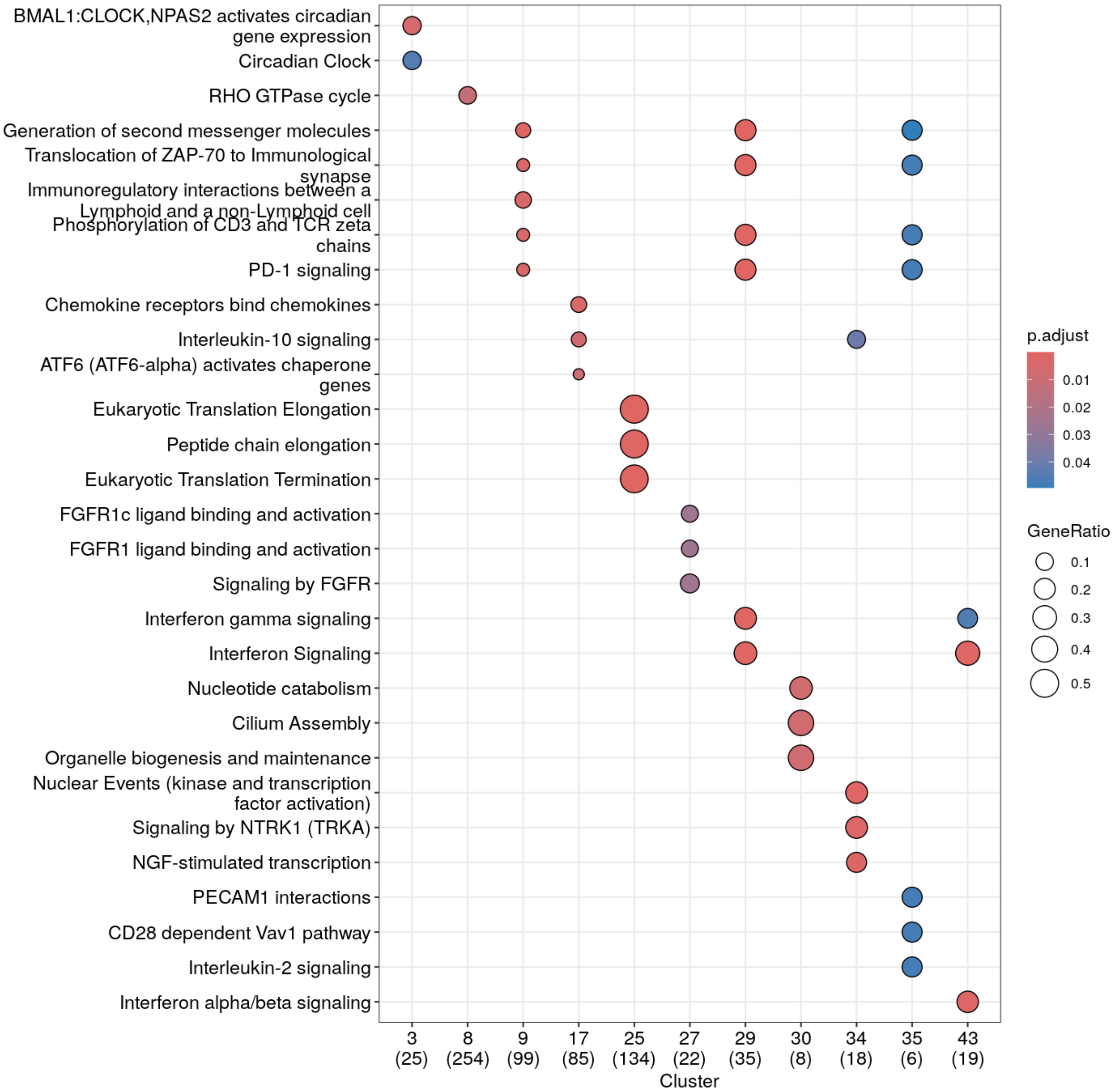


Figure 22: Reactome enrichment for modules in the CD14-monocyte consensus network.

14 Conclusions

This project presents a complete single-cell RNA-seq analysis of PBMC samples, progressing from raw sequencing data through quality control, cell-type annotation, differential expression, pathway enrichment, and cell-type–specific co-expression network analysis. The workflow successfully recovered the expected major PBMC lineages and produced consistent results across donors, providing a stable basis for downstream analyses.

Donor-aware pseudobulk differential expression analysis captured clear transcriptional divergence between lymphoid and myeloid populations. Complementary equivalence testing (TOST) helped distinguish genes with genuinely small expression differences from those that were simply not significant, allowing a clearer separation between contrast-specific markers and shared transcriptional programs across cell types.

Consensus co-expression network analysis further revealed structured modular organization within each cell type. Module-level enrichment analyses identified immune signaling pathways consistent with known PBMC biology, including interferon responses, inflammatory signaling, and adaptive immune activation programs. The networks also highlighted cell-type–specific features, such as strong innate immune activation in the monocyte network and adaptive signaling modules within the CD4 T-cell network.

Cell-type annotation was performed using a transparent marker-based scoring strategy; while this approach reliably recovers major PBMC lineages, more refined subtype classification and improved identification of ambiguous or contaminating populations could be achieved by integrating reference-based annotation methods in future iterations of the workflow.

Overall, the results demonstrate that integrating pseudobulk inference with consensus network analysis provides complementary perspectives on PBMC transcriptional organization. While enrichment-based interpretation should remain cautious due to the size of some modules and the limitations of over-representation analysis, the combined results form a coherent picture of cell-type–specific immune regulatory structure in the dataset.

References

[1] Hao Y, Stuart T, Kowalski MH, et al. *Dictionary learning for integrative, multimodal and scalable single-cell analysis.* Nature Biotechnology. 2023. doi:10.1038/s41587-023-01767-y.

[2] Satija R, Butler A, Hoffman P, Stuart T. *SeuratObject: Data Structures for Single Cell Data.* R package version 4.1.4. 2023.

[3] Liberzon A, Birger C, Thorvaldsdóttir H, Ghandi M, Mesirov JP, Tamayo P. *The Molecular Signatures Database (MSigDB) hallmark gene set collection.* Cell Systems. 2015.

[4] Godec J, Tan Y, Liberzon A, Tamayo P, Bhattacharya S, Butte AJ, Mesirov JP, Haining WN. *Compendium of Immune Signatures Identifies Conserved and Species-Specific Biology in Response to Inflammation.* Immunity. 2016.

[5] Zheng GX, Terry JM, Belgrader P, et al. *Massively parallel digital transcriptional profiling of single cells.* Cell. 2017;171(5):1202–1214.e15.

[6] Villani AC, Satija R, Reynolds G, et al. *Single-cell RNA-seq reveals new types of human blood dendritic cells, monocytes, and progenitors.* Science. 2017;356(6335).

[7] Stuart T, Butler A, Hoffman P, et al. *Comprehensive Integration of Single-Cell Data.* Cell. 2019;177(7):1888–1902.e21.

[8] Hao Y, Hao S, Andersen-Nissen E, et al. *Integrated analysis of multimodal single-cell data.* Nature Biotechnology. 2021;39: 856–865.

[9] Aran D, Looney AP, Liu L, et al. *Reference-based analysis of lung single-cell sequencing reveals a transitional profibrotic macrophage.* Nature Immunology. 2019;20(2):163–172.

[10] Inkala S, Fratello M, del Giudice G, Migliaccio G, Serra A, Greco D, Federico A. *MUUMI: an R package for statistical and network-based meta-analysis for multi-omics data integration.* BMC Bioinformatics. 2026;27:63.